

SOFTWARE ENGINEERING – LEVEL 3 · MINI-PROJECT

Software Requirements Specification (SRS)

Electronics Store Inventory Management System (ESIMS) · Version 1.0

Project Title	Electronics Store Inventory Management System (ESIMS)
Project ID	ESIMS-2026-Go7
Group Number	7
Team Members	C. Harshal Rajesh – PES1UG24CS118 Ayaan Ahmed – PES1UG24CS102
Course	UE24CS341A – Software Engineering
Semester	V Semester
Development Methodology	Agile
Document Version	1.0
Submission Format	PDF

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. External Interface Requirements
- 4. Analysis Models
- 5. System Features — Functional Requirements
- 6. Non-Functional Requirements
- 7. Security Requirements
- 8. Business / Domain Rules
- 9. Other Requirements — Database / Field Layout
- A. Appendix A — Glossary
- B. Appendix B — Requirement Traceability Matrix

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the **Electronics Store Inventory Management System (ESIMS)**, Version 1.0. ESIMS is a web-based application that enables an electronics store to manage its component catalogue, record stock-in and stock-out movements, monitor stock availability, and generate inventory reports and dashboards. This document describes the intended behaviour, scope, interfaces, constraints, and requirements against which the system will be designed, implemented, tested, and validated.

1.2 Document Conventions

- Every requirement is uniquely identified: FR-xxx (functional), NFR-xxx (non-functional), SEC-xxx (security) and UC-xx (use case).
- The keyword “**shall**” denotes a mandatory requirement.
- Priority is expressed as High, Medium or Low; each requirement lists a planned verification method.
- The document structure is adapted from IEEE Std 830-1998 and the PES University SE Level 3 submission guide.

1.3 Intended Audience and Reading Suggestions

This document is intended for the course faculty and evaluators, the project team members, and any future maintainers of the system. Readers seeking an overview should read Sections 1–2; those interested in interfaces and models should read Sections 3–4; the detailed requirements are in Sections 5–8; and supporting detail (field layout, glossary, traceability) is in Section 9 and the appendices.

Intended Users of the System

- **Administrator** — a trained, higher-privilege user who manages users, catalogue data and reports.
- **Staff** — a non-technical operational user who maintains components and records stock movements.
- **System** — automated, scheduled logic that derives availability status and raises low-stock alerts.

1.4 Product Scope

ESIMS v1.0 is a single-location (single-tenant) inventory system focused on component catalogue management, stock movement tracking, availability classification, reporting and an operational dashboard. The scope of the first release is summarised below.

In Scope	Out of Scope
Authentication & role-based access (Admin, Staff)	POS / billing / customer checkout
Component CRUD, search and filtering	Online payment / bank settlement
Category & supplier management	Barcode / RFID hardware integration
Stock-in / stock-out with validation	Purchase-order / procurement automation
Availability classification & low-stock alerts	Accounting / ERP / GST integration
Availability report + CSV/PDF export	Multi-branch / multi-warehouse deployment
Dashboard with KPIs and charts	AI demand forecasting
Audit log	Native mobile apps

1.5 References

- Jackfruit – Software Engineering Project Guidelines 2026.
- PES University SE Level 3 Student Submission Guide (Coffee Kiosk exemplar).
- IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications (structure adapted).

2. Overall Description

2.1 Product Perspective

ESIMS is a brand-new, standalone web application — not an add-on to any existing product. Internally it is organised in clear layers: the web UI talks to an API / controller layer, which calls the business logic (services), which reaches the relational database through a data-access layer, with helper utilities handling exports and notifications.

2.2 Product Functions

- Authenticate users and enforce roles.
- Maintain the component catalogue (CRUD + search / filter).
- Maintain categories and suppliers.
- Record stock-in / stock-out and keep quantities accurate.
- Classify availability and raise low-stock alerts.
- Generate and export availability reports.
- Present an inventory dashboard.
- Maintain an audit log.

2.3 User Classes and Characteristics

User class	Privileges	Characteristics
Administrator	Full access: user management; delete components/categories/suppliers; all reports and dashboard	Trained, higher privilege
Staff	Add/edit components; record stock movements; view reports	Non-technical, frequent operational use
System (scheduled/derived)	Recompute availability status; raise low-stock alerts	Automated, no UI

2.4 Operating Environment

- **Client:** modern web browser (Chrome, Edge, Firefox) on desktop / laptop.
- **Server:** any OS running Python / Django, served via WSGI / ASGI.
- **Database:** PostgreSQL 14+ (SQLite for local development).
- **Network:** HTTP / HTTPS over LAN or internet.

2.5 Design and Implementation Constraints

- Web application built with Python / Django + Django REST Framework (approved stack; MERN or another stack may be substituted with faculty approval).
- Relational database with referential integrity and transactions.
- Passwords stored only as salted hashes.
- HTTPS in any deployed environment.
- Source managed on GitHub with an automated CI pipeline.

2.6 Assumptions and Dependencies

- Users have valid accounts created by an administrator.
- The store operates a single location / warehouse (single-tenant) for v1.0.
- A stable browser and network connection are available.
- No real payment / POS integration is required for v1.0.

3. External Interface Requirements

3.1 User Interfaces

- Login screen.
- Inventory list screen with search / filter and availability badges.
- Add / Edit component form with inline field validation.
- Stock-in / stock-out transaction form.
- Category & supplier management screens.
- Availability report screen with filters and an export button.
- Dashboard with KPI cards and charts.
- Consistent, responsive Bootstrap layout; validation and error messages shown inline.

3.2 Hardware Interfaces

Standard desktop / laptop hardware only; no specialized devices. (A barcode scanner is a possible future enhancement and is out of scope for v1.0.)

3.3 Software Interfaces

- REST API (JSON) between the frontend and backend.
- Django ORM to PostgreSQL.
- CSV / PDF export library for reports.
- Optional email / notification service for low-stock alerts (mocked in v1.0).

3.4 Communication Interfaces

- HTTPS for all client–server communication.
- Standard HTTP status codes and JSON request / response payloads for the API.

4. Analysis Models

4.1 Use Case Diagram

Actors: Administrator, Staff, System. **Use cases:** Login, Manage Components, Search Inventory, Manage Categories / Suppliers, Record Stock Movement, Generate Availability Report, View Dashboard, Manage Users.

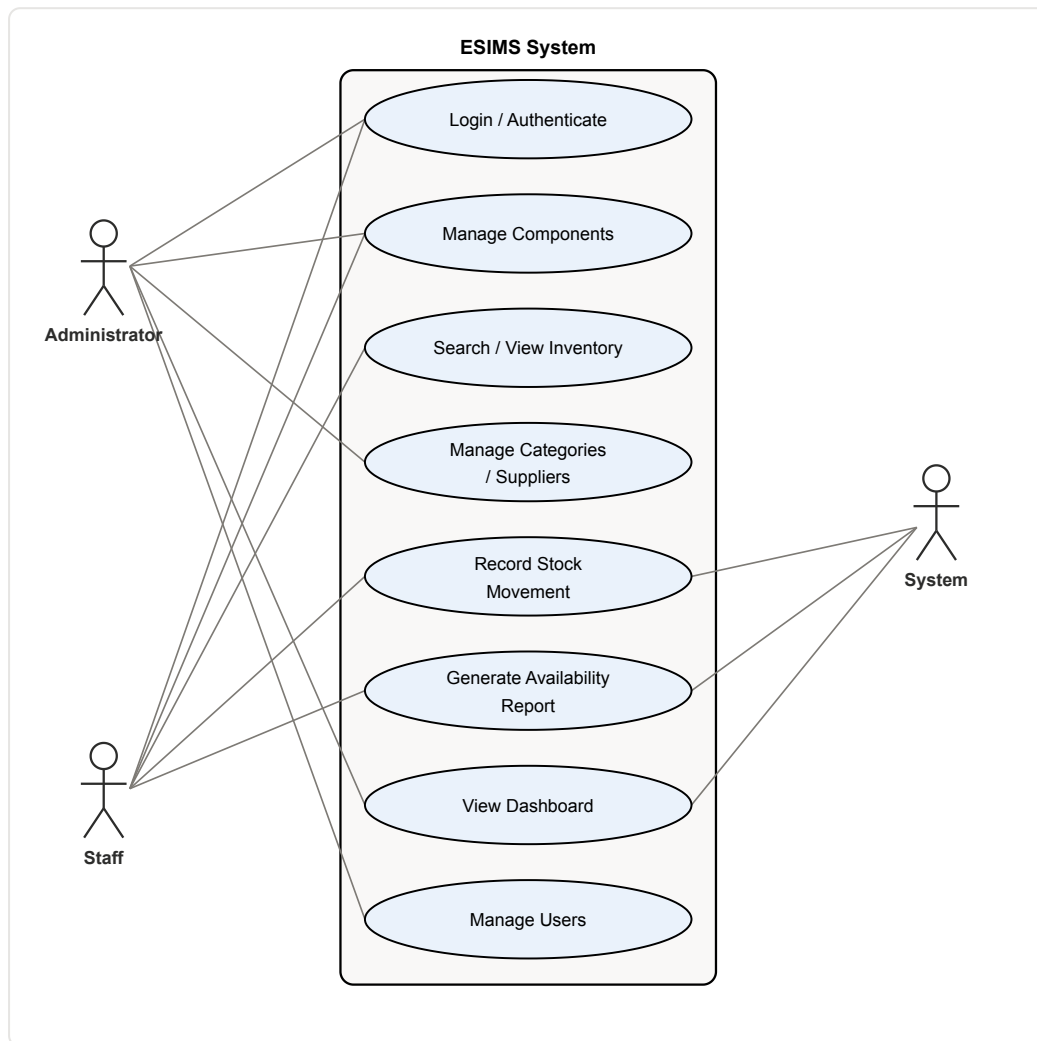


Figure 1 — ESIMS Use Case Diagram

4.2 Use Case Descriptions

UC ID	Use Case	Primary actor	Precondition	Outcome
UC-01	Login / Authenticate	Admin / Staff	Account exists	Authenticated session with role
UC-02	Add Component	Staff / Admin	Logged in	Component created with unique SKU
UC-03	Edit Component	Staff / Admin	Component exists	Component details updated

UC ID	Use Case	Primary actor	Precondition	Outcome
UC-04	Delete Component	Admin	Component exists	Component soft-deleted
UC-05	Search / View Inventory	Staff / Admin	Logged in	Filtered component list displayed
UC-06	Record Stock-in / Stock-out	Staff / Admin	Component exists	Quantity updated, transaction logged
UC-07	Manage Categories / Suppliers	Admin	Logged in	Category / supplier created / updated / removed
UC-08	Generate Availability Report	Staff / Admin	Logged in	Report displayed and exportable
UC-09	View Dashboard	Admin	Logged in	KPIs and charts displayed
UC-10	Manage Users	Admin	Admin logged in	Staff account created / disabled

4.3 Domain / ER Model

Entities: User, Category, Supplier, Component, StockTransaction, AuditLog.

Key relationships:

- Category 1 — * Component
- Supplier 1 — * Component
- Component 1 — * StockTransaction
- User 1 — * StockTransaction (performed_by)
- User 1 — * AuditLog

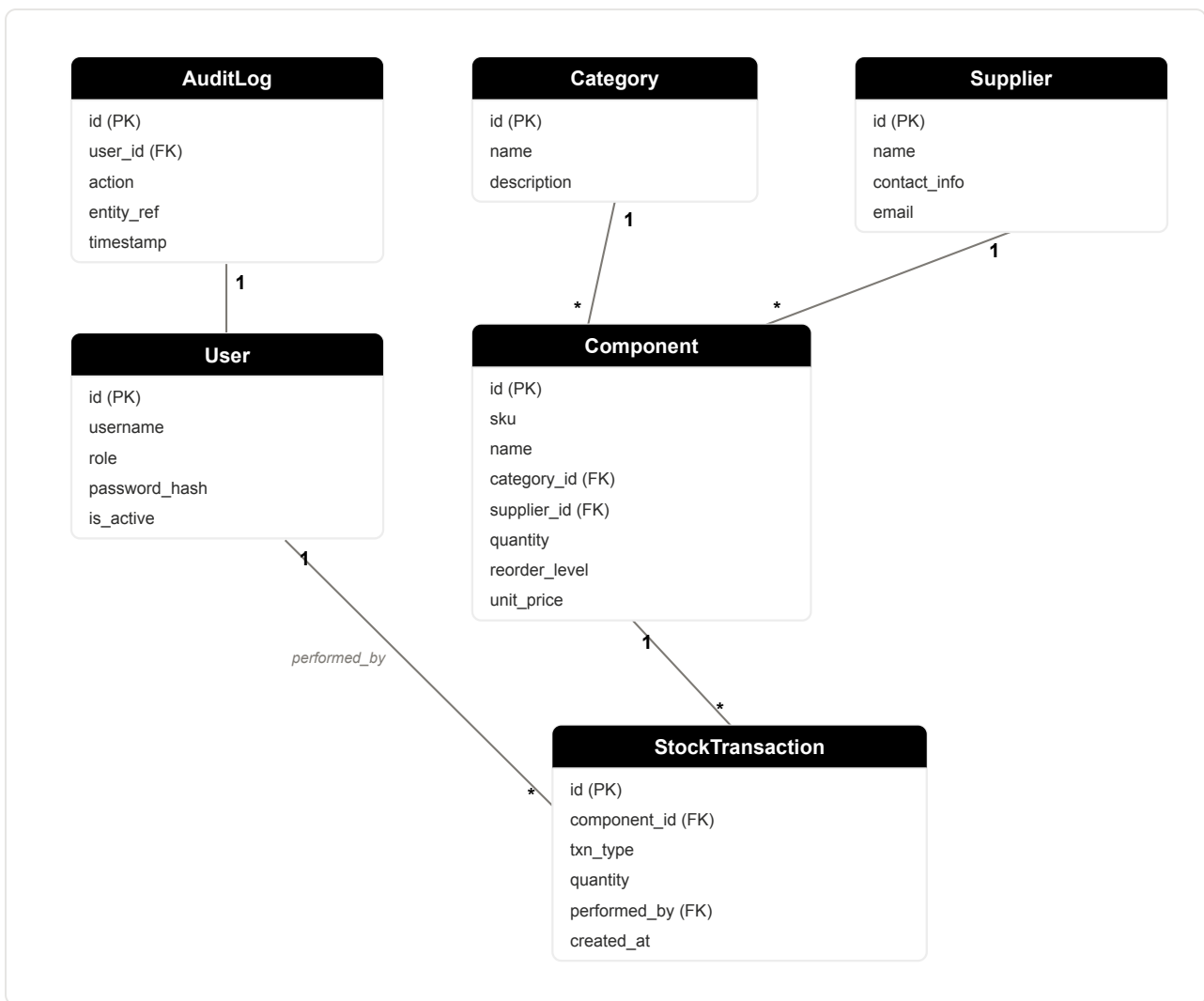


Figure 2 — ESIMS Entity-Relationship Diagram (1 — * denotes one-to-many)

5. System Features — Functional Requirements

ID	Requirement	Priority	Verification
FR-001	The system shall allow an administrator to create a user account with a unique username / email.	High	Functional test
FR-002	The system shall authenticate a user by username / email and password before granting access.	High	Functional test
FR-003	The system shall enforce role-based access so that Staff cannot perform Administrator-only actions.	High	Authorization test
FR-004	The system shall allow an authenticated user to add a component with name, SKU, category, supplier, unit price, quantity and reorder level.	High	Functional test
FR-005	The system shall reject a component whose SKU / code is not unique.	High	Negative test
FR-006	The system shall allow a user to edit the details of an existing component.	High	Functional test
FR-007	The system shall allow an administrator to delete (soft-delete) a component.	Medium	Functional test
FR-008	The system shall display a paginated, searchable list of components with current quantity and availability status.	High	System test
FR-009	The system shall allow filtering of components by category, supplier and availability status.	High	System test
FR-010	The system shall allow a user to record a stock-in transaction that increases a component's quantity.	High	Integration test
FR-011	The system shall allow a user to record a stock-out transaction and shall prevent a component's quantity from going below zero.	High	Negative + positive test
FR-012	The system shall automatically set availability to “Low Stock” when quantity $\leq$ reorder level and “Out of Stock” when quantity = 0.	High	Unit + system test
FR-013	The system shall allow an administrator to add, edit and delete categories.	Medium	Functional test
FR-014	The system shall allow an administrator to add, edit and delete suppliers.	Medium	Functional test
FR-015	The system shall generate an availability report of components, filterable by category, supplier and status.	High	System test
FR-016	The system shall allow the availability report to be exported as CSV and / or PDF.	Medium	Functional test
FR-017	The system shall display a dashboard showing total components, total stock value, low-stock count and out-of-stock count, with at least one chart.	High	System test
FR-018	The system shall record every stock transaction with component, type, quantity, user and timestamp.	High	Integration test

ID	Requirement	Priority	Verification
FR-019	The system shall log security-relevant events and deletions in an audit log.	Medium	System test

6. Non-Functional Requirements

ID	Category	Measurable requirement	Verification
NFR-001	Performance	95% of inventory list / search requests shall respond within 2 seconds for a catalogue of up to 5,000 components under the defined test load.	Performance test
NFR-002	Availability	The application shall be available at least 99% during store operating hours, excluding planned maintenance.	Operational review
NFR-003	Usability	A trained staff member shall be able to add a new component in no more than 3 primary screens.	Usability walkthrough
NFR-004	Reliability	Concurrent stock-out transactions shall never reduce a component's quantity below zero.	Concurrency / integration test
NFR-005	Maintainability	Business logic shall be organized into identifiable modules / services with automated unit tests.	Code review
NFR-006	Scalability	The system shall support at least 5,000 components and 10 concurrent users without functional degradation.	Load test
NFR-007	Portability	The web UI shall function on the latest versions of Chrome, Edge and Firefox.	Cross-browser test
NFR-008	Testability	Core services shall expose testable interfaces independent of the UI.	Unit / integration test

7. Security Requirements

ID	Security requirement	Validation approach
SEC-001	Passwords shall be stored only as salted hashes, never as plaintext.	Code / configuration review
SEC-002	Role-based access shall prevent Staff from accessing Administrator functions (delete, user management).	Authorization tests
SEC-003	The system shall validate and sanitize all user-supplied input to prevent SQL injection and XSS.	Negative / security tests
SEC-004	Authenticated sessions / tokens shall expire, and invalid or expired credentials shall be rejected.	Security test
SEC-005	State-changing requests shall be protected against CSRF.	Security test
SEC-006	Security-relevant events (failed login, unauthorized access, deletions) shall be logged.	System test

8. Business / Domain Rules

- A component's quantity shall never be negative.
- Every component shall have a unique SKU / code and belong to exactly one category.
- Reorder level shall be a non-negative integer.
- A stock-out shall be allowed only if sufficient quantity is available.
- Only an Administrator may delete components, categories, suppliers or user accounts.
- A category or supplier referenced by at least one component shall not be hard-deleted.
- Availability status is **derived, not manually set**: Out of Stock (qty = 0), Low Stock (0 < qty ≤ reorder level), In Stock (qty > reorder level).

9. Other Requirements — Database / Field Layout

Field	Length	Data type	Description	Mandatory
user_id	36	UUID / String	Unique user identifier	Y
username	50	String	Unique login name	Y
password_hash	128	String	Salted password hash	Y
role	20	Enum (Admin / Staff)	Access role	Y
category_id	36	UUID / String	Unique category identifier	Y
category_name	60	String	Category name (unique)	Y
supplier_id	36	UUID / String	Unique supplier identifier	Y
supplier_name	100	String	Supplier name	Y
component_id	36	UUID / String	Unique component identifier	Y
sku	40	String	Unique component code	Y
component_name	100	String	Display name	Y
unit_price	10,2	Decimal	Unit price	Y
quantity	10	Integer	Current stock quantity	Y
reorder_level	10	Integer	Low-stock threshold	Y
availability_status	15	Enum	In / Low / Out of stock (derived)	Y
txn_id	36	UUID / String	Stock transaction identifier	Y
txn_type	10	Enum (IN / OUT)	Transaction type	Y
txn_quantity	10	Integer	Quantity moved	Y
txn_timestamp	—	DateTime	When the transaction occurred	Y

Appendix A — Glossary

Term	Meaning
SKU	Stock Keeping Unit — unique code identifying a component
Reorder level	Quantity threshold at / below which a component is considered low on stock
Stock-in	Transaction that increases quantity (restock / receipt)
Stock-out	Transaction that decreases quantity (issue / sale / usage)
RBAC	Role-Based Access Control
KPI	Key Performance Indicator
CRUD	Create, Read, Update, Delete
Availability status	Derived state: In Stock / Low Stock / Out of Stock

Appendix B — Requirement Traceability Matrix (initial)

Req ID	Description	Use Case	Module / Owner	Planned Test Case
FR-002	Authenticate user	UC-01	Auth (Ayaan)	UT-AUTH-01 / ST-AUTH-01
FR-004	Add component	UC-02	Inventory (Ayaan)	UT-INV-01 / ST-INV-01
FR-008	List / search inventory	UC-05	Inventory (Ayaan)	ST-INV-02
FR-011	Stock-out with validation	UC-06	Stock (Harshal)	UT-STK-01 / IT-STK-01
FR-012	Availability classification	UC-06 / UC-08	Stock (Harshal)	UT-STK-02
FR-015	Availability report	UC-08	Reporting (Harshal)	ST-RPT-01
FR-017	Dashboard KPIs	UC-09	Dashboard (Harshal)	ST-DSH-01
SEC-002	RBAC enforcement	UC-01	Auth (Ayaan)	ST-SEC-01
NFR-001	Performance	—	API layer	PERF-01

The RTM will be extended with architecture references, code-file references and executed test results during the Design and Implementation & Testing phases.

PES UNIVERSITY, BANGALORE

Department of Computer Science and
Engineering

SOFTWARE ENGINEERING – LEVEL 3 · MINI-PROJECT

Software Architecture and Design Specification (SAD)

Electronics Store Inventory Management System (ESIMS) · Version 1.0

Project Title	Electronics Store Inventory Management System (ESIMS)
Project ID	ESIMS-2026-Go7
Group Number	7
Team Members	C. Harshal Rajesh – PES1UG24CS118 Ayaan Ahmed – PES1UG24CS102
Course	UE24CS341A – Software Engineering
Semester	V Semester
Development Methodology	Agile
Document Version	1.0
Submission Format	PDF

Document Information

Project	Electronics Store Inventory Management System (ESIMS)
Document	Software Architecture and Design Specification
Version	1.0
Authors	C. Harshal Rajesh (PES1UG24CS118), Ayaan Ahmed (PES1UG24CS102)
Date	02-10-2026
Status	Baseline – submitted for review

Revision History

Version	Date	Author	Change summary
1.0	02-10-2026	Harshal, Ayaan	First baseline of the Software Architecture and Design Specification for the Level 3 submission

Approvals

Role	Name	Signature / Date
Course Faculty	Dr. Sapna V M	
Team Member	C. Harshal Rajesh	
Team Member	Ayaan Ahmed	

Table of Contents

1.	Introduction
2.	Document Overview
3.	Architecture 3.1 Goals & Constraints · 3.2 Stakeholders & Concerns · 3.3 Component (UML) Diagram · 3.4 Component Descriptions · 3.5 Chosen Architecture Pattern and Rationale · 3.6 Technology Stack & Data Stores · 3.7 Risks & Mitigations · 3.8 Traceability to Requirements · 3.9 Security Architecture
4.	Design 4.1 Design Overview · 4.2 UML Sequence Diagrams · 4.3 API Design · 4.4 Error Handling, Logging & Monitoring · 4.5 UX Design · 4.6 Open Issues & Next Steps
5.	Appendices 5.1 Glossary · 5.2 References · 5.3 Tools

1. Introduction

1.1 Purpose

This document sets out the architecture and design of the Electronics Store Inventory Management System (ESIMS) v1.0. The SRS says *what* the system has to do. This document explains *how* we have split it into parts, how those parts talk to each other, where the data lives and how the main workflows run. We will build from it, and the Software Test Plan uses it to decide what to test at each level.

1.2 Scope

The design covers everything marked "In Scope" in SRS Section 1.4: login and roles, the component catalogue, categories and suppliers, stock-in / stock-out, availability classification and low-stock alerts, the availability report with CSV / PDF export, the dashboard and the audit log. Out-of-scope items (POS / billing, barcode hardware, multi-branch, ERP / GST, forecasting) are left out on purpose. Where we have left room for them later, Section 4.6 says so.

1.3 Audience

Course faculty and evaluators, the two of us on the team, the QA role described in the Test Plan, and anyone who maintains the code later.

1.4 Definitions

Term	Meaning
SAD	Software Architecture and Design Specification (this document)
DRF	Django REST Framework, the library used for the JSON API
ORM	Object-Relational Mapper (Django's model layer)
ViewSet	DRF class that groups the API actions for one resource
Service	Plain Python module holding the business rules for one area. Views call services and services call models
Soft delete	Marking a row as deleted (<code>is_deleted = true</code>) instead of removing it
STRIDE	Threat categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
ADR	Architecture Decision Record. Our decisions are listed as D1–D7 in Section 3.5
SKU, RBAC, KPI	As defined in SRS Appendix A

2. Document Overview

2.1 How to use this document

Section 3 is the architecture: the big picture, the components, why we picked this pattern, the stack, the risks, how each requirement maps to a component, and the security architecture. Section 4 is the design: sequence diagrams for the two most important flows, the API, error handling and the screens. If you only want to know where a requirement is handled, go straight to the traceability table in Section 3.8.

Requirement IDs (FR-, NFR-, SEC-) are the ones in the SRS. Test IDs (UT-, IT-, ST-, PERF-) are defined in the Software Test Plan. BR-n means the n-th business rule in SRS Section 8, counted from the top.

2.2 Related Documents

Document	Where
ESIMS Software Requirements Specification (SRS) v1.0	Part 1 of this combined PDF
ESIMS Software Test Plan (STP) v1.0	Part 3 of this combined PDF
Requirements Traceability Matrix (RTM)	SRS Appendix B (initial), SAD Section 3.8 (requirement → design → code), STP Section 13 (requirement → test cases)
Project repository	github.com/harshal-prozone/Electronics-Store-Inventory-Management-System

3. Architecture

3.1 Goals & Constraints

These are the things that shaped the design most. Each one points back to the SRS.

Goal / constraint	What it means for the design	Source
Stock must never go negative, even under concurrent use	Every stock movement runs in one database transaction that locks the component row first. A CHECK constraint on <code>quantity >= 0</code> is the last line of defence.	FR-011, NFR-004, BR-1
Availability is derived, never typed in	Only <code>AvailabilityService.classify()</code> sets the status. It runs whenever quantity or reorder level changes.	FR-012, BR-7
Role checks happen on the server	Permission classes on every API view. Hiding buttons in the UI is a convenience, never the control.	FR-003, SEC-002
Business logic is testable without the UI	Rules live in service modules that can be unit-tested with plain inputs. Views stay thin.	NFR-005, NFR-008
Fast list / search on 5,000 items	Server-side pagination and indexes on <code>sku</code> , <code>name</code> , <code>category_id</code> , <code>status</code> .	NFR-001, NFR-006
99% availability in store hours	Docker restart policies, a health check endpoint and daily database backups.	NFR-002
Approved stack and tooling	Python 3.12, Django 5, DRF, PostgreSQL 14, Bootstrap 5, Chart.js, Git / GitHub, Jenkins CI, Docker, pytest, SonarQube (course guidelines).	SRS 2.5
Security basics	Salted password hashing, session expiry, CSRF tokens, ORM-only queries, output escaping, HTTPS.	SEC-001 – SEC-006

3.2 Stakeholders & Concerns

Stakeholder	Main concerns	Where addressed
Store Administrator (owner / manager)	Accurate stock figures, control over who can delete or manage users, quick overview of low stock and stock value	3.5 (D5, D6), 3.9, 4.5 (Dashboard)
Store Staff	Fast search, few clicks to record stock, clear errors when an action is refused	4.3, 4.4, 4.5
Course faculty / evaluators	Requirements traced to design and tests, readable UML, security considered	3.3, 3.8, 3.9, 4.2
Development team (Harshal, Ayaan)	Clear module boundaries so each of us can work alone, stable API contracts, easy local setup	3.4, 4.3 contract rules, 3.6
Future maintainers	Consistent layout across modules, tests next to code, documented decisions	3.5, 4.1

3.3 Component (UML) Diagram

Figure 1 shows the Django apps that make up ESIMS and how they depend on each other. Arrows point from the module that uses to the module being used. No module reaches into another module's models directly. It goes through that module's service instead.

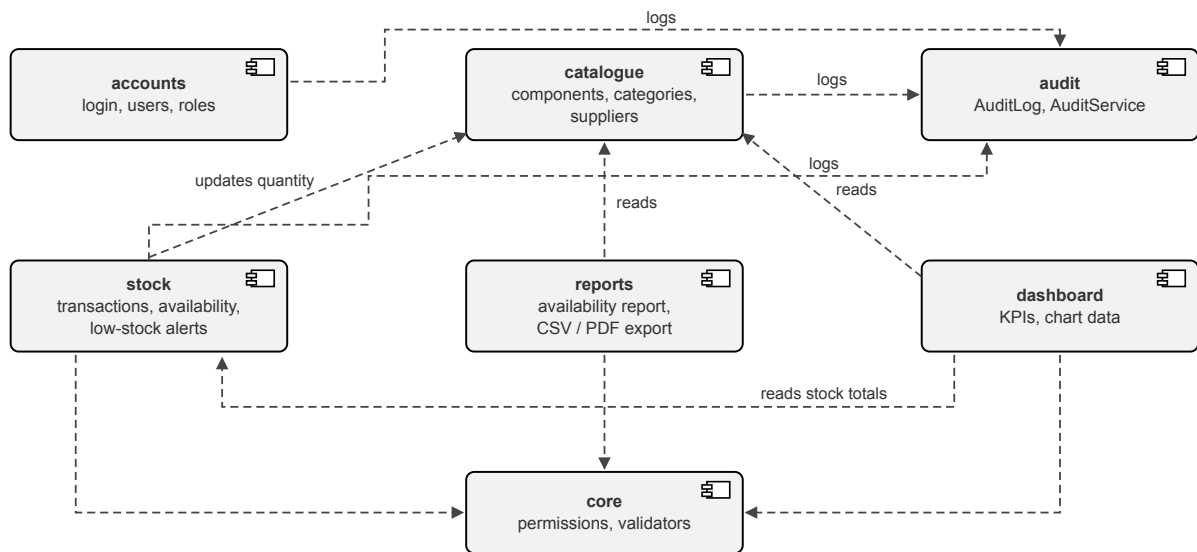


Figure 1 — UML component diagram (Django apps and their dependencies)

Figure 2 is the class diagram for the domain model. The classes match the entities in the SRS ER model (SRS Section 4.3). StockService is shown because it is the only class allowed to create a StockTransaction and change a quantity.

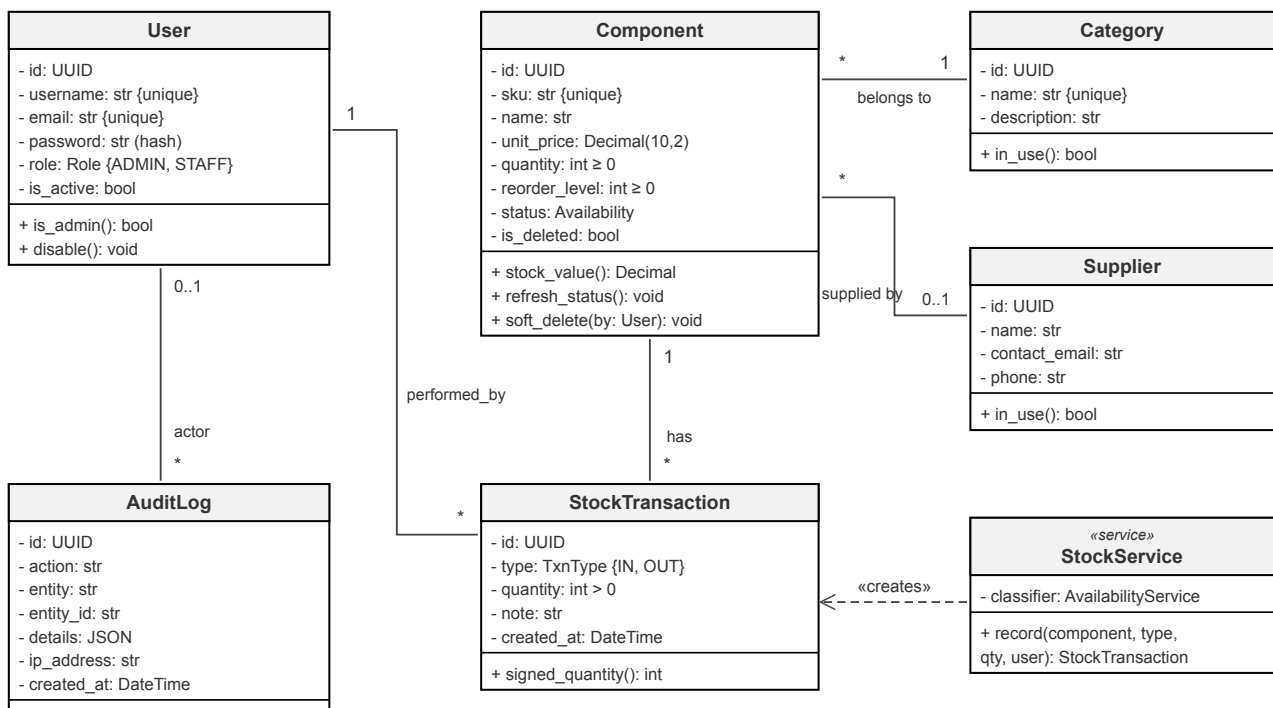


Figure 2 — UML class diagram (domain model)

3.4 Component Descriptions

Component (app)	Responsibilities	Depends on	Arch. ref	Owner
accounts	Login / logout, session handling, lockout, user create / disable, roles (Admin, Staff)	core, audit	AUTH-01, AUTH-02	Ayaan
catalogue	Component CRUD + soft delete, search / filter / pagination, categories, suppliers	core, audit, stock (AvailabilityService)	INV-01, CAT-01	Ayaan
stock	Stock-in / stock-out, row locking, transaction history, availability classification, low-stock alerts	catalogue, audit, core	STK-01, STK-02	Harshal
reports	Availability report with filters, CSV and PDF export	catalogue, core	RPT-01	Harshal
dashboard	KPI aggregation (counts, stock value), chart data by category	catalogue, stock, core	DSH-01	Harshal
audit	AuditLog model, <code>AuditService.log()</code> , admin-only log view	core	AUD-01	Shared
core	Permission classes (<code>IsAdmin</code> , <code>IsStaffOrAdmin</code>), base templates, validators, error format, health check	—	CORE-01	Shared
Web UI	Django templates + Bootstrap 5, small JS modules that call the API, Chart.js	REST API	UI-01	Ayaan (layout), each owner for own screens
PostgreSQL	Durable storage, referential integrity, CHECK and UNIQUE constraints	—	DB-01	Harshal (setup)

Every app has the same file layout, so either of us can find our way around the other's code: `models.py` · `services.py` · `serializers.py` · `views.py` · `urls.py` · `permissions.py` · `templates/` · `tests/`.

Work division. Each of us owns complete features end to end, not one layer. Harshal owns stock and availability, reports and export, and the dashboard. He also looks after the Jenkins / Docker pipeline, PostgreSQL setup and the performance test. Ayaan owns login and user management, the component catalogue, and categories and suppliers. He also looks after the Bootstrap layout, the security tests and API reviews.

3.5 Chosen Architecture Pattern and Rationale

We chose a **layered architecture, built as a modular monolith**: one Django project, split into small apps, with a strict top-down flow from UI to API to services to models to the database (Figure 3). A request always enters at the top and moves down one layer at a time. The API layer decides whether the *user* is allowed to try an action. The service layer decides whether the action is allowed by the *business rules*.

We did look at microservices and at a separate React front end. For a two-person team, one store and one database they would mostly add deployment work without solving a problem we actually have, so we rejected them for v1.0. Clear module boundaries keep things simple now and still let us split modules out later if the store goes multi-branch.

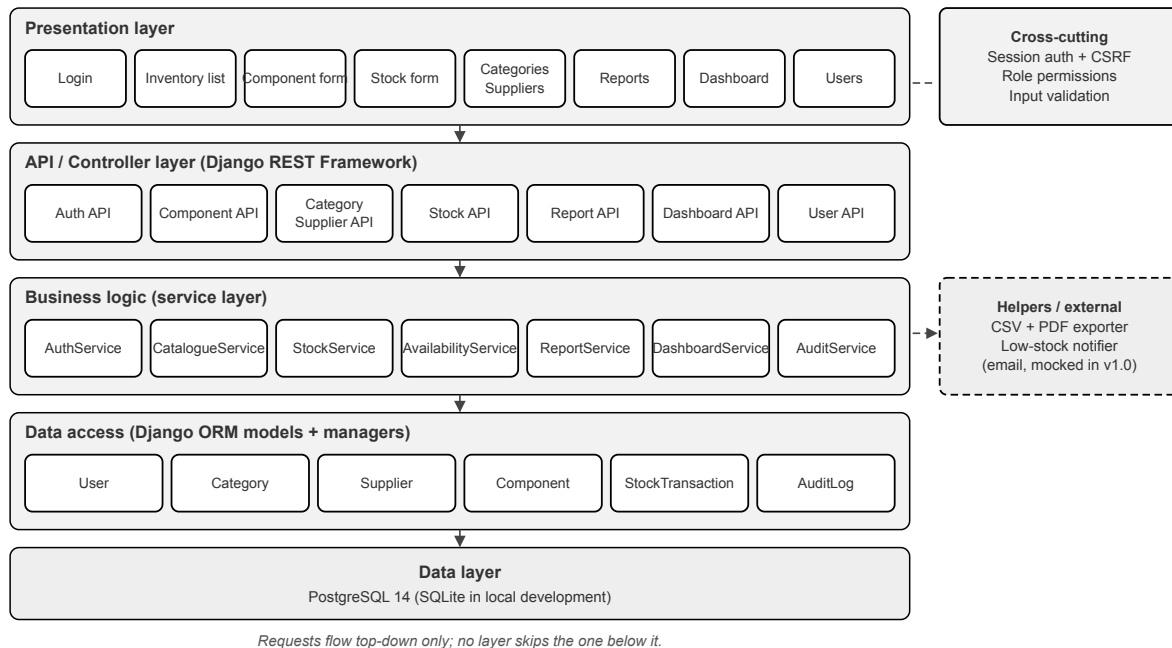


Figure 3 — ESIMS layered architecture

#	Decision	Why we chose it	Alternative considered
D1	Layered modular monolith (Django apps)	Small team, single store, one database. Easy to deploy and debug.	Microservices: too much overhead for v1.0
D2	Server-rendered Bootstrap pages that call a JSON REST API with <code>fetch()</code>	One codebase. The API stays reusable for a future mobile app. No separate build step.	React SPA: nicer UX but doubles the front-end work
D3	Service layer between views and models	Keeps the rules (stock checks, classification, delete guards) in one place we can test.	Fat models / fat views: harder to unit-test
D4	Session authentication with CSRF, not JWT	Browser-only clients. Django sessions give expiry and CSRF protection out of the box (SEC-004, SEC-005).	JWT: needs our own refresh and revocation logic
D5	Row lock (<code>SELECT ... FOR UPDATE</code>) for stock movements	Simplest reliable way to stop two stock-outs from overselling.	Optimistic version column: more retries for users
D6	Soft delete for components; block delete for in-use categories / suppliers	Keeps transaction history and reports consistent (FR-007, BR-6).	Hard delete with cascade: loses history
D7	Quantity is read-only on the edit form	Quantity only changes through stock transactions, so every change leaves a trace (FR-018).	Free edit: breaks the audit trail

3.6 Technology Stack & Data Stores

Layer	Technology	Why
Presentation	Django templates, Bootstrap 5, vanilla JS, Chart.js	Responsive, no build step, dashboard charts (FR-017)
API	Django REST Framework	Serializers give validation; permission classes give RBAC
Business logic	Python 3.12 service modules	Plain functions, easy to unit-test
Database	PostgreSQL 14 (SQLite for quick local runs)	Row-level locks, CHECK constraints, reliable transactions
Exports	Python csv module, ReportLab	CSV and PDF report export (FR-016)
Web server	nginx 1.25 + gunicorn	HTTPS termination, static files, worker processes
Packaging / CI	Docker, Docker Compose, Jenkins, SonarQube	Same image in test and deployment; quality gate on every push
Testing	pytest, pytest-django, coverage.py, Selenium, Locust, Bandit, pip-audit, OWASP ZAP	See the Test Plan

Figure 4 shows how this runs. nginx terminates HTTPS and serves static files, gunicorn runs Django, and PostgreSQL keeps its data on a named volume so containers can be rebuilt without losing anything. Jenkins runs on every push to GitHub and only builds an image when lint, tests and the security scan pass.

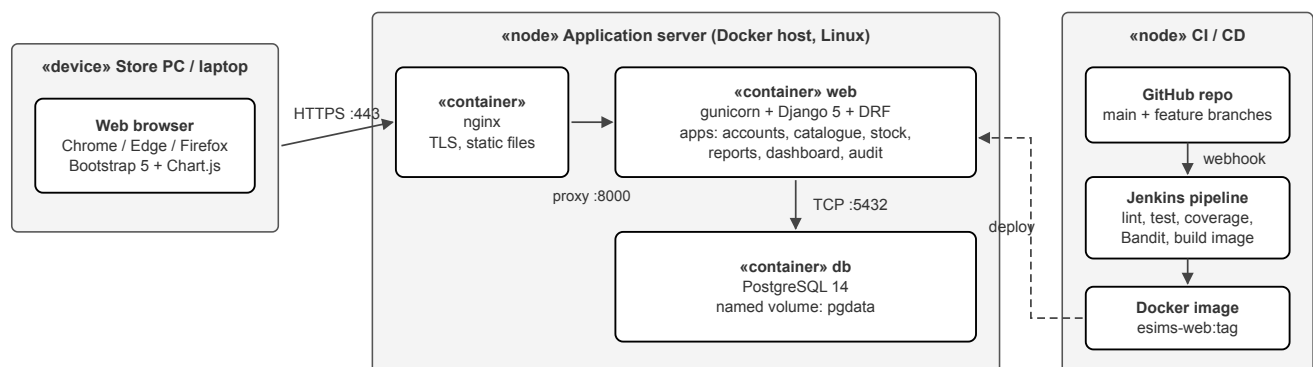


Figure 4 – Deployment diagram

Data store: tables and constraints

Field sizes follow SRS Section 9. These constraints are enforced in the database itself, not only in Python, so a bug in one view cannot quietly corrupt data.

Table	Key columns	Constraints / indexes
accounts_user	id, username, email, password, role, is_active	UNIQUE(username), UNIQUE(email); role IN ('ADMIN', 'STAFF')
catalogue_category	id, name, description	UNIQUE(lower(name))
catalogue_supplier	id, name, contact_email, phone	INDEX(name)
catalogue_component	id, sku, name, category_id, supplier_id, unit_price, quantity, reorder_level, status, is_deleted, deleted_at	UNIQUE(sku); CHECK(quantity ≥ 0); CHECK(reorder_level ≥ 0); CHECK(unit_price ≥ 0); FK category / supplier ON DELETE PROTECT; INDEX(name), INDEX(status), INDEX(category_id)
stock_stocktransaction	id, component_id, type, quantity, note, performed_by_id, created_at	CHECK(quantity > 0); type IN ('IN', 'OUT'); FK component PROTECT; INDEX(component_id, created_at)

Table	Key columns	Constraints / indexes
audit_auditlog	id, actor_id, action, entity, entity_id, details (JSON), ip_address, created_at	Insert-only from the application; INDEX(created_at), INDEX(action)

- SKU is trimmed and upper-cased before the uniqueness check, so `mcu-ard-uno` and `MCU-ARD-UNO` count as the same code.
- `status` is stored for fast filtering but is always recomputed by `AvailabilityService`. It is never accepted from the client.
- Money uses `Decimal(10,2)`, never floats. Stock value is $\sum \text{quantity} \times \text{unit_price}$ over non-deleted components, computed in the database.

3.7 Risks & Mitigations

Risk	Mitigation
Two users issue the last units at the same time and stock goes negative	Row lock in <code>StockService</code> (D5) plus the <code>CHECK</code> constraint. Tested by IT-STK-02 on PostgreSQL.
Locking behaves differently on SQLite and PostgreSQL	CI and the test deployment use PostgreSQL. SQLite is only for quick local runs.
Modules slowly become tangled	Apps only call each other through services. Code review on every PR checks for direct cross-app model access.
One of us changes an API and breaks the other's screens	Contract rules in Section 4.3. Breaking changes need a PR approved by both of us.
List, report or dashboard gets slow as data grows	Indexes, pagination, <code>select_related</code> and database aggregates. PERF-01 checks the 2-second target.
Data lost when containers are rebuilt or the disk fails	Named Docker volume and a daily <code>pg_dump</code> copied off the machine. Restore steps written in the README.
Single server goes down during store hours	Docker restart: <code>unless-stopped</code> , health check endpoint, and the backup above for recovery.

3.8 Traceability to Requirements

This extends SRS Appendix B with the architecture and design references from this document. Code file references follow the module layout in Section 3.4. Test IDs are defined in the Test Plan.

Req ID	Component (arch. ref)	Design ref	Code / file ref	Test case ID	System test ID
FR-001	AUTH-02	Class: User; API /api/users/	accounts/services.py	UT-USR-01	ST-USR-01
FR-002	AUTH-01	API /api/auth/login/	accounts/views.py	UT-AUTH-01	ST-AUTH-01
FR-003	AUTH-02	core permission classes	core/permissions.py	IT-USR-01	ST-SEC-01
FR-004	INV-01	Seq: add component (Fig. 8)	catalogue/services.py	UT-INV-01	ST-INV-01
FR-005	INV-01	UNIQUE(sku), SKU normalisation	catalogue/services.py	UT-INV-02	IT-INV-01
FR-006	INV-01	PATCH /api/components/{id}/	catalogue/views.py	UT-INV-07	ST-INV-03
FR-007	INV-01	Soft delete (D6)	catalogue/models.py	UT-INV-11	ST-INV-05
FR-008	INV-01	Pagination, indexes	catalogue/views.py	IT-INV-11	ST-INV-02
FR-009	INV-01	Filter params	catalogue/filters.py	UT-INV-16	ST-INV-02
FR-010	STK-01	Activity (Fig. 5)	stock/services.py	UT-STK-03	ST-STK-01
FR-011	STK-01	Seq: stock-out (Fig. 7), row lock	stock/services.py	UT-STK-01	IT-STK-01
FR-012	STK-02	State machine (Fig. 6)	stock/availability.py	UT-STK-02	ST-STK-01
FR-013	CAT-01	ON DELETE PROTECT	catalogue/services.py	UT-CAT-01	ST-CAT-01
FR-014	CAT-01	Supplier class	catalogue/services.py	UT-SUP-01	ST-CAT-01
FR-015	RPT-01	/api/reports/availability/	reports/services.py	UT-RPT-01	ST-RPT-01
FR-016	RPT-01	Export endpoint	reports/exporters.py	UT-RPT-03	ST-RPT-01
FR-017	DSH-01	/api/dashboard/summary/	dashboard/services.py	UT-DSH-02	ST-DSH-01
FR-018	STK-01	StockTransaction class	stock/models.py	IT-STK-01	ST-STK-01
FR-019	AUD-01	AuditService	audit/services.py	IT-AUTH-03	ST-SEC-04
NFR-001	CORE-01, DB-01	Indexes, pagination (4.1)	catalogue/views.py	—	PERF-01
NFR-002	CORE-01	Health check, restart policy (4.4)	core/views.py	—	ST-NFR-03
NFR-003	UI-01	Screen flow (Fig. 9)	templates/	—	ST-NFR-02
NFR-004	STK-01	Row lock + CHECK (4.1)	stock/services.py	IT-STK-02	IT-STK-02
NFR-005	all apps	Same app layout (3.4)	*/services.py	—	ST-NFR-04
NFR-006	DB-01	Indexes, aggregates	catalogue/views.py	—	PERF-01
NFR-007	UI-01	Bootstrap 5, no plugins	templates/	—	ST-NFR-01
NFR-008	all apps	Service layer (D3)	*/tests/	UT-STK-02	ST-NFR-04
SEC-001	AUTH-01	PBKDF2 hashing	settings.py	UT-AUTH-03	ST-SEC-03
SEC-002	AUTH-02	Permission classes	core/permissions.py	IT-INV-07	ST-SEC-01
SEC-003	CORE-01	Serializers, ORM, escaping	*/serializers.py	IT-INV-12	ST-SEC-03
SEC-004	AUTH-01	Session timeout, logout	settings.py	IT-AUTH-02	ST-SEC-05
SEC-005	CORE-01	CSRF middleware	settings.py	IT-SEC-01	ST-SEC-02
SEC-006	AUD-01	Audit events (3.9)	audit/services.py	IT-AUTH-03	ST-SEC-04

3.9 Security Architecture

There are three trust boundaries: the browser and nginx (public network, HTTPS only), nginx and the Django container (internal Docker network), and Django and PostgreSQL (internal network, database port never published). Everything that crosses the first boundary is treated as untrusted. The STRIDE table lists what could go wrong at each point and what stops it.

Threat (STRIDE)	Example for ESIMS	Control in the architecture	Req
Spoofing	Someone guesses a staff password or reuses a stolen session	PBKDF2-SHA256 salted hashes; logout after 5 failed logins in 15 min; Secure + HttpOnly + SameSite cookies; 30-min idle timeout	SEC-001, SEC-004
Tampering	Changing quantity or status directly through the API; SQL injection in search	Quantity and status are read-only fields; quantity only changes through StockService; ORM only, no raw SQL; CSRF tokens on POST / PATCH / DELETE; DB CHECK constraints	SEC-003, SEC-005, FR-018
Repudiation	A staff member denies issuing stock or an admin denies deleting an item	Every stock movement stores the user and time; AuditLog is insert-only and records actor, action, IP and time	SEC-006, FR-019
Information disclosure	Stack traces shown to users; staff viewing admin data; traffic sniffed on the store Wi-Fi	DEBUG off in deployment; generic 500 messages; RBAC on every endpoint; HTTPS only with HSTS; DB not reachable from outside	SEC-002
Denial of service	Flooding the login endpoint; asking for huge pages	Account logout; nginx rate limit on /api/auth/login/; max page size 100; gunicorn timeouts	SEC-004, NFR-001
Elevation of privilege	Staff calling an admin-only endpoint by typing its URL	IsAdmin permission class checked on the server for every admin action; users cannot change their own role; admins cannot disable themselves	SEC-002, FR-003

Security controls by requirement

Concern	Design	Req
Password storage	Django PBKDF2-SHA256 with per-user salt; minimum length 8 plus Django's common-password validator	SEC-001
Authorisation	IsAdmin / IsStaffOrAdmin permission classes on every ViewSet; templates check role only to hide links	SEC-002
Input handling	DRF serializers validate type, length and range; ORM only; templates auto-escape; CSV cells starting with = + - are prefixed with ' so spreadsheets do not run them as formulas	SEC-003
Sessions	30-minute idle timeout; logout and account disable end the user's sessions	SEC-004
CSRF	Django CSRF middleware; JS sends the X-CSRFToken header on unsafe requests	SEC-005
Audit	LOGIN_FAILED, UNAUTHORIZED_ACCESS, *_DELETED, USER_DISABLED, STOCK_IN / STOCK_OUT logged with actor, IP and time	SEC-006
Pipeline	Bandit, pip-audit and SonarQube run in Jenkins; a high-severity finding fails the build	SEC-003

4. Design

4.1 Design Overview

Inside each module the design follows the same layers as the architecture. Views and serializers deal with HTTP and validation only. Services hold the rules and are the only code that writes to the database. Models declare the tables and constraints. This is what lets us unit-test the rules without a browser (NFR-008).

Concurrency and transactions. `StockService.record()` runs inside `transaction.atomic()` and loads the component with `select_for_update()`. The quantity check, the update, the transaction row and the status change all commit together or not at all. The database CHECK constraint means that even if someone later bypasses the service by mistake, a negative quantity is still rejected (NFR-004).

Performance. List and report queries use `select_related('category', 'supplier')` to avoid N+1 queries, paginate on the server and use the indexes in Section 3.6. Dashboard KPIs are database aggregates, one query each. With 5,000 components this keeps responses well inside the 2-second target in NFR-001.

Availability rules. Figure 5 shows the stock movement workflow and Figure 6 shows the three availability states. A component only changes state through a stock movement or a change to its reorder level (FR-012).

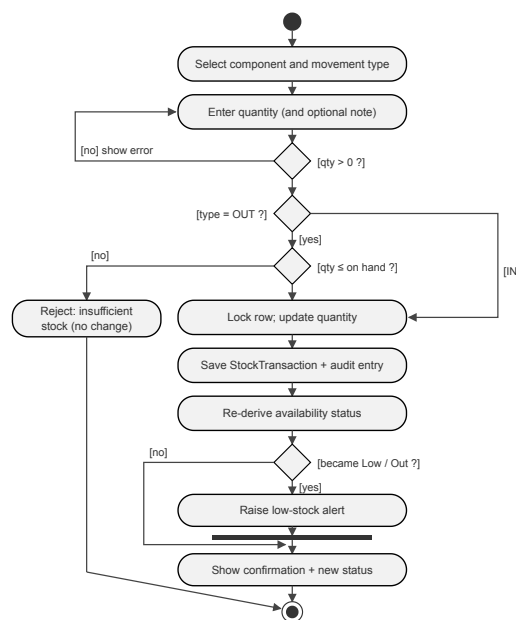


Figure 5 — UML activity diagram: stock-in / stock-out workflow

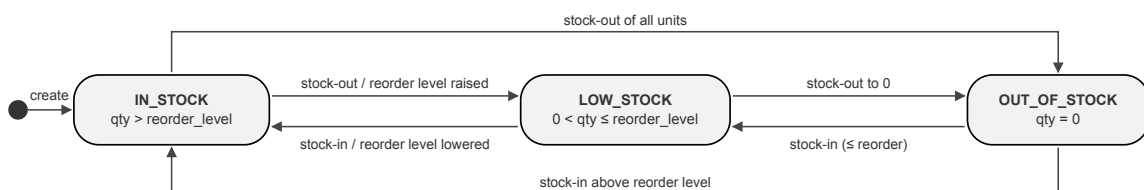


Figure 6 — UML state machine: component availability

```
def classify(qty, reorder_level):
    if qty == 0: return OUT_OF_STOCK
    if qty <= reorder_level: return LOW_STOCK
    return IN_STOCK
```

A low-stock alert is raised only when the status *changes* into `LOW_STOCK` or `OUT_OF_STOCK`, not on every stock-out while the item is already low. Otherwise staff would get the same alert over and over.

4.2 UML Sequence Diagrams

4.2.1 Record stock-out (UC-06)

This is the flow that matters most for data integrity. The row lock is taken before reading the quantity, so two people issuing the last few units at the same moment cannot both succeed.

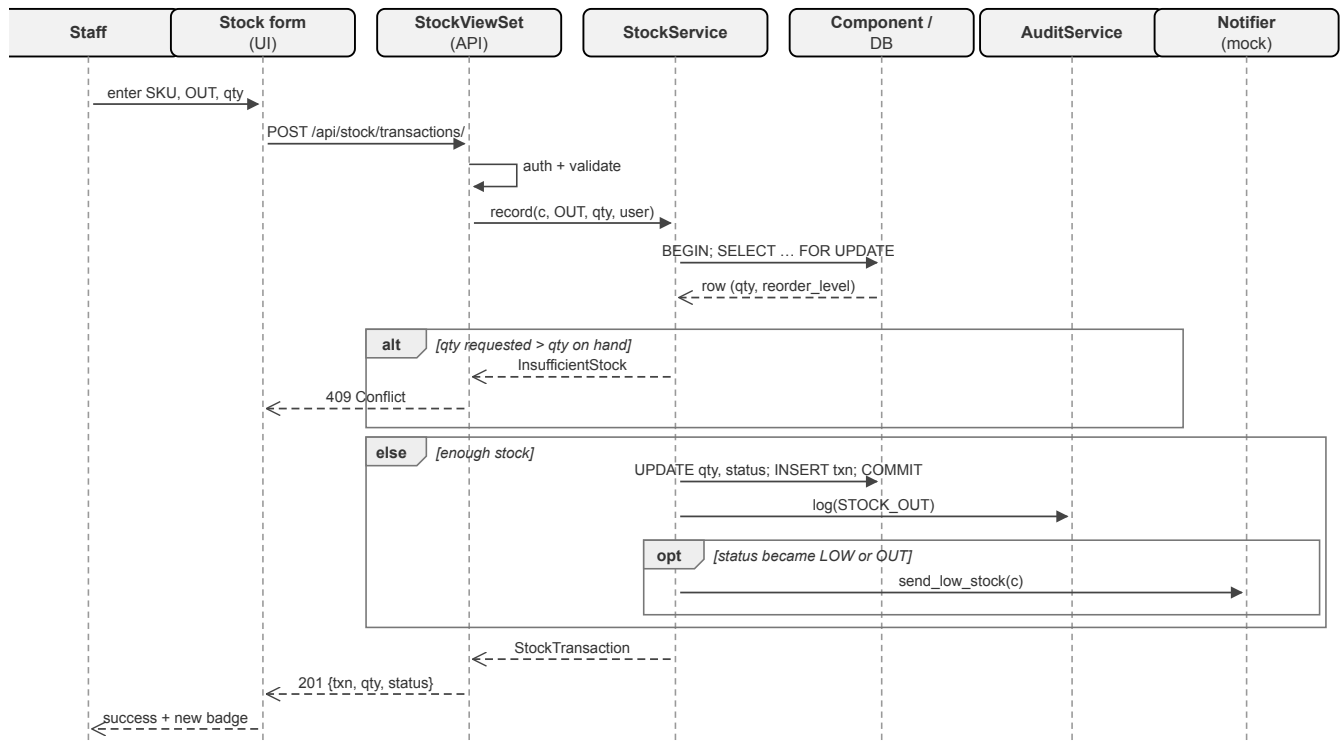


Figure 7 — UML sequence diagram: record stock-out

4.2.2 Add component (UC-02)

The SKU is normalised and checked before anything is saved. The database UNIQUE constraint catches the rare case where two people add the same SKU at the same moment.

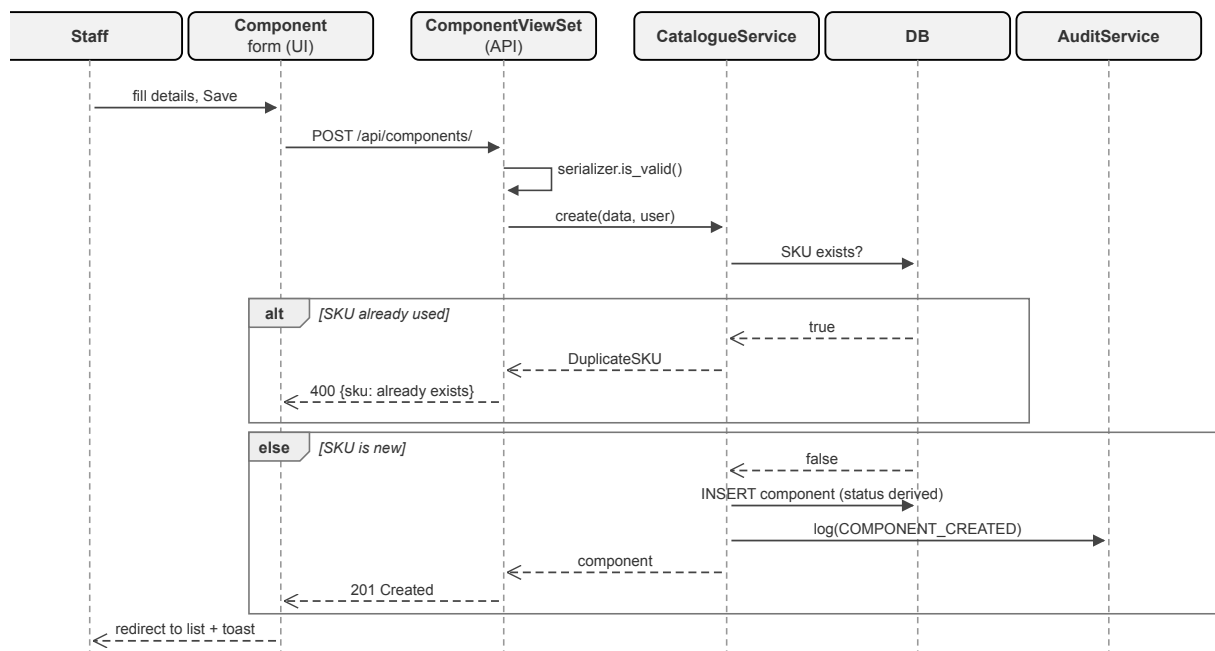


Figure 8 — UML sequence diagram: add component with duplicate-SKU check

4.3 API Design

All endpoints live under `/api/`, take and return JSON (except file exports), and need a logged-in session unless marked Public. Unsafe methods (POST, PATCH, DELETE) must carry the CSRF token. Below are the full interface definitions for the four main components, followed by the complete endpoint list.

Stock component — record a stock movement

Field	Definition
Endpoint	<code>/api/stock/transactions/</code>
Method	POST
Access	Staff, Admin
Request	<code>{"component": "uuid", "type": "IN" "OUT", "quantity": 3, "note": "Issued to repair bench"}</code>
Response	<code>201 {"txn_id": "uuid", "component": "uuid", "quantity": 7, "status": "LOW_STOCK", "created_at": "2026-10-02T10:15:00+05:30"}</code>
Errors	400 VALIDATION_ERROR (quantity $\leq$ 0, bad type) · 401 not logged in · 403 CSRF · 404 component not found or deleted · 409 INSUFFICIENT_STOCK

Catalogue component — add a component

Field	Definition
Endpoint	<code>/api/components/</code>
Method	POST
Access	Staff, Admin
Request	<code>{"sku": "MCU-ARD-UNO", "name": "Arduino Uno R3", "category": "uuid", "supplier": "uuid", "unit_price": "650.00", "quantity": 25, "reorder_level": 5}</code>
Response	201 component object including the derived status
Errors	400 VALIDATION_ERROR (missing field, negative price / reorder level) · 400 DUPLICATE_SKU · 401 · 403 CSRF

Auth component — log in

Field	Definition
Endpoint	<code>/api/auth/login/</code>
Method	POST
Access	Public
Request	<code>{"username": "staff1", "password": "....."}</code>
Response	200 <code>{"user": "staff1", "role": "STAFF"}</code> + session cookie
Errors	400 missing fields · 401 INVALID_CREDENTIALS (same message whether username or password is wrong) · 423 ACCOUNT_LOCKED or ACCOUNT_DISABLED

Reports component — export availability report

Field	Definition
Endpoint	<code>/api/reports/availability/export/</code>
Method	GET
Access	Staff, Admin
Request	Query: <code>format=csv pdf</code> , optional category, supplier, status
Response	200 file (text/csv or application/pdf) with filters and generation time in the header

Field	Definition
Errors	400 UNSUPPORTED_FORMAT or bad filter · 401

Complete endpoint list

Method	Endpoint	Who	Input	Success	Errors
POST	/api/auth/login/	Public	{username, password}	200 {user, role} + session cookie	400, 401, 423
POST	/api/auth/logout/	Any user	—	204	401
GET	/api/users/	Admin	?search	200 [users]	401, 403
POST	/api/users/	Admin	{username, email, password, role}	201 user	400 (duplicate / weak), 403
PATCH	/api/users/{id}/	Admin	{is_active, role}	200 user	400 (own account), 403, 404
GET	/api/components/	Staff, Admin	?search, category, supplier, status, page	200 {count, next, results[]}	400 (bad filter), 401
POST	/api/components/	Staff, Admin	{sku, name, category, supplier, unit_price, quantity, reorder_level}	201 component	400 (validation / duplicate SKU)
GET	/api/components/{id}/	Staff, Admin	—	200 component	404
PATCH	/api/components/{id}/	Staff, Admin	any editable field except quantity, status	200 component	400, 404
DELETE	/api/components/{id}/	Admin	—	204 (soft delete)	403, 404
GET / POST	/api/categories/	GET: Staff, Admin · POST: Admin	{name, description}	200 / 201	400 (duplicate), 403
PATCH / DELETE	/api/categories/{id}/	Admin	{name, description}	200 / 204	403, 404, 409 (in use)
GET / POST	/api/suppliers/	GET: Staff, Admin · POST: Admin	{name, contact_email, phone}	200 / 201	400, 403
PATCH / DELETE	/api/suppliers/{id}/	Admin	{...}	200 / 204	403, 404, 409 (in use)
POST	/api/stock/transactions/	Staff, Admin	{component, type, quantity, note}	201 {txn, quantity, status}	400, 404, 409
GET	/api/stock/transactions/	Staff, Admin	?component, type, from, to	200 [transactions]	400
GET	/api/reports/availability/	Staff, Admin	?category, supplier, status	200 {generated_at, filters, rows[], totals}	400
GET	/api/reports/availability/export/	Staff, Admin	?format=csv pdf + filters	200 file	400

Method	Endpoint	Who	Input	Success	Errors
GET	/api/dashboard/summary/	Admin	—	200 {total_components, stock_value, low_count, out_count, by_category[]}	403
GET	/api/audit-logs/	Admin	?action, from, to	200 [entries]	403
GET	/api/health/	Public	—	200 {status: ok, db: ok}	503 if the DB is unreachable

Internal service interfaces

Modules call each other through these Python functions, never through each other's models. Their signatures are treated as a contract, the same as the REST API.

Service	Signature	Raises
StockService	record(component_id, type, quantity, user, note='') → StockTransaction	InsufficientStock, ValidationError, NotFound
AvailabilityService	classify(quantity, reorder_level) → Status	—
CatalogueService	create(data, user) → Component · update(id, data, user) · soft_delete(id, user)	DuplicateSKU, ValidationError, NotFound
ReportService	build(filters) → Report · export(report, format) → bytes	UnsupportedFormat
DashboardService	summary() → dict	—
AuditService	log(actor, action, entity, entity_id, details, ip)	—

Contract rules

- **Status codes:** 200 / 201 / 204 success; 400 validation; 401 not logged in or session expired; 403 wrong role or missing CSRF; 404 not found (also for soft-deleted items); 409 business-rule conflict; 423 locked or disabled account.
- **Read-only fields:** id, quantity (after create), status, timestamps. If the client sends them, they are ignored.
- **Pagination:** 25 per page by default, 100 at most.
- **Versioning:** v1.0 has no URL prefix. A breaking change goes under /api/v2/ and the old endpoint stays until both modules have moved over. We agreed never to change a contract quietly once the other person's code depends on it.

4.4 Error Handling, Logging & Monitoring

Error handling. Services raise domain exceptions (InsufficientStock, DuplicateSKU, InUseError). One DRF exception handler in `core` turns them into a single error format, so the UI can show field errors next to the right input:

```
{"error": {"code": "INSUFFICIENT_STOCK", "message": "Only 2 units of MCU-ARD-UNO are available.", "fields": {"quantity": ["Exceeds available stock"]}}
```

Situation	HTTP	Code	What the user sees
Missing or invalid field	400	VALIDATION_ERROR	Red message under the field
SKU already used	400	DUPLICATE_SKU	'This SKU already exists'
Not logged in / session expired	401	NOT_AUTHENTICATED	Sent to the login page
Wrong role	403	FORBIDDEN	'You don't have permission to do this'
Item not found or deleted	404	NOT_FOUND	'This item no longer exists'

Situation	HTTP	Code	What the user sees
Not enough stock	409	INSUFFICIENT_STOCK	Available quantity shown; nothing saved
Category / supplier in use	409	IN_USE	'Used by 12 components'
Account locked or disabled	423	ACCOUNT_LOCKED	'Try again in 15 minutes' / 'Contact your admin'
Anything unexpected	500	SERVER_ERROR	'Something went wrong, please try again'. Full trace goes to the server log only

Logging. Application logs are written as one JSON line per event with time, level, request ID, user ID and path. Passwords, session IDs and CSRF tokens are never logged. Security events go to the AuditLog table as well (Section 3.9), so an admin can look at them in the app without server access.

Monitoring. GET /api/health/ checks the app and the database connection. Docker uses it to restart an unhealthy container and an uptime check calls it every 5 minutes (NFR-002). We keep an eye on four numbers: 5xx error rate, p95 response time for the inventory list, failed logins per hour, and the result of the nightly backup. Jenkins shows build, test, coverage and scan status for every push.

4.5 UX Design

The screens come from SRS Section 3.1. Admins land on the dashboard and staff land on the inventory list, because that is what each of them opens most. A top navigation bar links everything else, and Admin-only links are not shown to Staff.

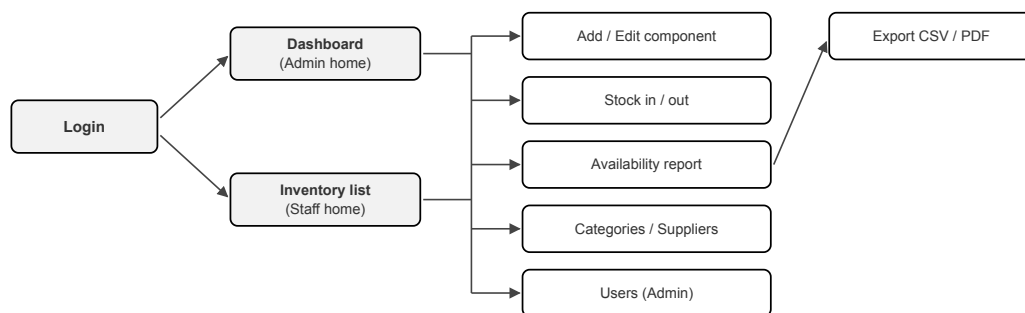


Figure 9 – Screen navigation map

Screen	Main elements	Reqs
Login	Username / email, password, one error message that doesn't say which of the two was wrong	FR-002
Inventory list	Search box, category / supplier / status filters, paginated table, coloured badges (green In, amber Low, red Out), Add button	FR-008, FR-009
Add / Edit component	One form with inline validation; quantity read-only when editing; Save / Cancel	FR-004 – FR-006, NFR-003
Stock in / out	Component picker (search by SKU or name), IN / OUT toggle, quantity, note; current quantity shown before submit	FR-010, FR-011
Categories / Suppliers	Table with inline add / edit; delete disabled with a tooltip when in use	FR-013, FR-014
Availability report	Filters, results table with totals, Export CSV / PDF buttons	FR-015, FR-016
Dashboard	Four KPI cards, bar chart of stock by category, latest low-stock items	FR-017
Users	List, create staff, enable / disable	FR-001

Adding a component takes three screens: login, inventory list, then the add form (NFR-003). Status badges use a text label as well as colour, so they still make sense for colour-blind users. Every form field has a visible label, and the whole UI works with keyboard only.

4.6 Open Issues & Next Steps

Item	Status / plan
Real email for low-stock alerts	v1.0 uses Django's console email backend. Switch to SMTP once the store gives us a mail account.
OpenAPI / Swagger documentation	Generate from the serializers with drf-spectacular during Sprint 2 so the endpoint list stays in sync with the code.
Barcode scanner input	Out of scope for v1.0. The stock form already accepts SKU typing, so a USB scanner that types the SKU would work without changes.
Multi-branch support	Out of scope. Would need a <code>store_id</code> on components and transactions. The module boundaries let us add this without a rewrite.
Design updates during implementation	If the code moves away from this design, we update the diagrams and Section 3.8 in the same PR, so the documents keep matching the code.

5. Appendices

5.1 Glossary

Term	Meaning
SKU	Stock Keeping Unit, the unique code for a component
Reorder level	Quantity at or below which a component counts as low on stock
Stock-in / Stock-out	Transactions that increase / decrease quantity
RBAC	Role-Based Access Control
CSRF	Cross-Site Request Forgery
HSTS	HTTP Strict Transport Security
KPI	Key Performance Indicator
p95	95th percentile response time
STRIDE, ADR, DRF, ORM	See Section 1.4

5.2 References

- ESIMS Software Requirements Specification v1.0, Group 7 (Part 1 of this PDF).
- Jackfruit – Software Engineering Project Guidelines 2026, PES University.
- SE Level 3 Student Submission Guide (Coffee Kiosk exemplar), PES University.
- ISO/IEC/IEEE 42010:2011, Architecture description.
- IEEE Std 1016-2009, Software Design Descriptions.
- OWASP Top 10 (2021) and OWASP ASVS 4.0.
- Django 5 and Django REST Framework documentation.

5.3 Tools

Purpose	Tool
UML diagrams	draw.io / PlantUML style, exported as SVG
API documentation	drf-spectacular (Swagger UI), Postman collection
Version control and tracking	Git, GitHub (Issues, Projects board, pull requests)
CI / CD and containers	Jenkins, Docker, Docker Compose
Code quality and security	SonarQube, flake8, Bandit, pip-audit, OWASP ZAP
Testing	pytest, coverage.py, Selenium, Locust

PES UNIVERSITY, BANGALORE

Department of Computer Science and
Engineering

SOFTWARE ENGINEERING – LEVEL 3 · MINI-PROJECT

Software Test Plan (STP)

Electronics Store Inventory Management System (ESIMS) · Version 1.0

Project Title	Electronics Store Inventory Management System (ESIMS)
Project ID	ESIMS-2026-G07
Group Number	7
Team Members	C. Harshal Rajesh – PES1UG24CS118 Ayaan Ahmed – PES1UG24CS102
Course	UE24CS341A – Software Engineering
Semester	V Semester
Development Methodology	Agile
Document Version	1.0
Submission Format	PDF

Document Information

Project	Electronics Store Inventory Management System (ESIMS)
Document	Software Test Plan
Version	1.0
Authors	C. Harshal Rajesh (PES1UG24CS118), Ayaan Ahmed (PES1UG24CS102)
Date	02-10-2026
Status	Baseline – submitted for review

Revision History

Version	Date	Author	Change summary
1.0	02-10-2026	Harshal, Ayaan	First baseline of the Software Test Plan for the Level 3 submission

Approvals

Role	Name	Signature / Date
Course Faculty	Dr. Sapna V M	
Team Member	C. Harshal Rajesh	
Team Member	Ayaan Ahmed	

Table of Contents

1.	Introduction
2.	Test Items
3.	Features to be Tested
4.	Features Not to be Tested
5.	Test Approach / Strategy
5.1	Security Validation
6.	Test Environment
7.	Test Schedule
8.	Test Deliverables
9.	Roles and Responsibilities
10.	Risks and Mitigation
11.	Assumptions & Dependencies
12.	Suspension & Resumption Criteria
13.	Test Case Management & Traceability (RTM and test cases)
14.	Test Metrics & Reporting
15.	Approvals

1. Introduction

Purpose. This document is the test plan for the Electronics Store Inventory Management System (ESIMS) v1.0. It sets out what we will test and at which level, the approach, the environment and data, who does what and when, and the actual test cases. It also explains how the security and non-functional requirements will be checked, so nothing in the SRS is left without a test.

Scope. All 19 functional requirements, 8 non-functional requirements and 6 security requirements in the SRS, across the ten use cases UC-01 to UC-10. Testing runs at unit (services), integration (API + database) and system (the whole application through the browser) level, plus security and performance testing. Out-of-scope features from SRS 1.4 are not tested (Section 4).

References. ESIMS SRS v1.0 and ESIMS SAD v1.0 (Parts 1 and 2 of this PDF); Jackfruit – Software Engineering Project Guidelines 2026; SE Level 3 Student Submission Guide (Coffee Kiosk exemplar); IEEE Std 829-2008, Software Test Documentation; OWASP Web Security Testing Guide v4.2.

Definitions.

Term	Meaning
STP	Software Test Plan (this document)
RTM	Requirements Traceability Matrix
UT / IT / ST	Unit / Integration / System test case ID prefixes
PERF	Performance test case
SUT	System Under Test: the ESIMS build being tested
UAT	User Acceptance Testing
SRS, SAD	Software Requirements Specification, Software Architecture and Design Specification

2. Test Items

Item	What is tested	Version / build	Owner
accounts module	Login, sessions, lockout, user management, roles	ESIMS v1.0 builds from main	Ayaan
catalogue module	Components, categories, suppliers, search / filter	v1.0	Ayaan
stock module	Stock-in / out, row locking, availability, low-stock alerts	v1.0	Harshal
reports module	Availability report, CSV / PDF export	v1.0	Harshal
dashboard module	KPI aggregation, chart data	v1.0	Harshal
audit and core modules	Audit log, permission classes, error handler, health check	v1.0	Shared
REST API	All endpoints in SAD Section 4.3	v1.0 (no URL prefix)	Shared
Deployment	Docker images, nginx HTTPS configuration	Docker Compose file in repo	Harshal

3. Features to be Tested

Every requirement in the SRS is in scope for testing. The short names below are ours. The full wording is in the SRS. Section 13 shows which test cases cover each one.

Req ID	Feature	Req ID	Feature
FR-001	Admin creates user accounts	FR-018	Record every stock transaction

Req ID	Feature	Req ID	Feature
FR-002	Authenticate user	FR-019	Audit log of security events
FR-003	Role-based access	NFR-001	95% of list / search ≤ 2 s
FR-004	Add component	NFR-002	$\geq 99\%$ availability in store hours
FR-005	Reject duplicate SKU	NFR-003	Add a component in ≤ 3 screens
FR-006	Edit component	NFR-004	Concurrent stock-outs never go negative
FR-007	Soft-delete component (Admin)	NFR-005	Business logic in identifiable modules
FR-008	Paginated, searchable list	NFR-006	5,000 components, 10 concurrent users
FR-009	Filter by category / supplier / status	NFR-007	Latest Chrome, Edge, Firefox
FR-010	Record stock-in	NFR-008	Testable core services
FR-011	Stock-out, never below zero	SEC-001	Salted password hashes
FR-012	Automatic Low / Out of Stock status	SEC-002	RBAC blocks Staff from Admin functions
FR-013	Manage categories	SEC-003	Input validated against SQLi / XSS
FR-014	Manage suppliers	SEC-004	Sessions expire; bad credentials rejected
FR-015	Availability report with filters	SEC-005	CSRF protection
FR-016	Export report CSV / PDF	SEC-006	Security events logged
FR-017	Dashboard KPIs and chart		

4. Features Not to be Tested

Feature	Reason
POS / billing, payments, barcode / RFID hardware, ERP / GST, multi-branch, forecasting, mobile apps	Out of scope for v1.0 (SRS 1.4)
Real email delivery of low-stock alerts	The notifier uses Django's console backend in v1.0. We only check that it is called at the right time with the right data
Browser versions older than the current release	NFR-007 only covers the latest Chrome, Edge and Firefox
Internals of Django, DRF, PostgreSQL and nginx	Third-party software, assumed tested by its maintainers. We test our use and configuration of it

5. Test Approach / Strategy

Levels.

Level	What it checks	How	Tools
Unit	Service functions and validators on their own (stock rules, classification, SKU handling, report and KPI maths)	White-box. Written with the code in the same sprint. Django test DB; notifier mocked	pytest, pytest-django, coverage.py
Integration	API + permissions + database together: status codes, transactions, audit entries, concurrency	Black-box at the API with DRF APIClient; PostgreSQL in CI for the locking tests	pytest, DRF APIClient, Postman
System	Complete use cases through the browser, as a user would do them	Black-box, scripted manual runs + Selenium for the main flows	Selenium, manual checklist
Acceptance (UAT)	The store's main tasks work for a real user	A non-team user runs the UAT checklist (ST-NFR-02 plus the main system tests)	Manual

Types.

Type	Scope
Functional	All FRs, positive and negative cases for each
Regression	Full unit + integration suite on every push and pull request in Jenkins
Performance / load	NFR-001 and NFR-006: 5,000 components, 10 concurrent users (Locust)
Concurrency	NFR-004: parallel stock-outs on PostgreSQL
Security	Section 5.1
Usability	NFR-003 walkthrough with a new user
Compatibility	NFR-007 on the latest Chrome, Edge and Firefox
Static analysis	SonarQube, flake8 and Bandit on every build

Test design technique. We mainly use **equivalence partitioning with boundary values**. Quantity and reorder level are where most bugs hide, so for each rule we test just below, on and just above the boundary (quantity 0, 1, reorder_level and reorder_level + 1). Every functional requirement has at least one positive and one negative case. Unit tests are written by the module owner. Integration and system tests for a module are reviewed by the *other* team member, so nobody signs off only their own work.

Criteria	
Entry criteria	Build compiles and starts in Docker; migrations run on a fresh PostgreSQL DB; seed data loads; the module's unit tests pass in Jenkins; test environment from Section 6 is up
Exit criteria	100% of planned test cases executed; 100% of High-priority cases pass and $\geq 95\%$ of all cases pass; 0 open Critical or High defects; $\geq 80\%$ line coverage on service modules; PERF-01 within target; no open high-severity security findings; RTM shows every requirement covered and passed
Pass / fail of a case	Pass when the actual result matches the expected result with no unexpected side effects (extra rows, wrong audit entries). Anything else is a fail and gets a defect linked to the test ID

5.1 Security Validation

Security testing follows the STRIDE threat model in SAD Section 3.9. Each security requirement is checked in more than one way: automated tests in the pipeline, scanners, and a manual attempt to break it.

- **Password handling:** passwords stored only as salted PBKDF2 hashes, never in logs or API responses; weak passwords refused (SEC-001).
- **Access control:** every Admin-only endpoint called with a Staff session and with no session, both through the UI and by typing the URL / calling the API directly (SEC-002).
- **Injection and XSS:** SQL injection strings in search and filters, `<script>` in text fields, and CSV formula injection in exported reports (SEC-003).
- **Sessions and login:** idle timeout, logout, disabled accounts, lockout after 5 failed attempts, and cookie flags Secure / HttpOnly / SameSite (SEC-004).
- **CSRF:** unsafe requests without a token and from another origin are refused (SEC-005).
- **Audit trail:** failed logins, 403s, deletions and stock movements all appear in the audit log with actor, IP and time (SEC-006).
- **Transport:** HTTP redirects to HTTPS, HSTS header present, database port not reachable from outside the Docker network.
- **Automated scanning:** Bandit (code), pip-audit (dependencies) and SonarQube on every build; OWASP ZAP baseline scan against the test deployment each sprint from Sprint 3.
- **Manual penetration test:** a short session in Sprint 4 trying to edit quantity or status directly, reuse a logged-out session, and reach another user's data.

Security req	Validation method	Test cases
SEC-001	Unit test on stored hashes, code / config review, scan	UT-AUTH-03, UT-USR-03, ST-SEC-03
SEC-002	Negative API and UI tests with a Staff session	IT-INV-07, IT-CAT-01, IT-USR-01, IT-DSH-01, ST-SEC-01, ST-INV-05
SEC-003	Injection / XSS inputs, CSV injection, scanners	IT-INV-02, IT-INV-05, IT-INV-12, UT-RPT-04, ST-SEC-03
SEC-004	Session expiry, lockout, cookie flags	IT-AUTH-02, IT-AUTH-04, UT-USR-04, ST-SEC-05
SEC-005	Requests without a token / from another origin	IT-SEC-01, ST-SEC-02
SEC-006	Audit log checks after each security test	IT-AUTH-03, IT-USR-01, ST-SEC-04

Security test cases

These are the cross-cutting security cases. Security cases that belong to one use case (for example IT-INV-07 or IT-INV-12) are listed with that use case in Section 13.3.

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
IT-SEC-01	Integration	SEC-005	POST without CSRF token	Logged in	POST /api/components / with no X-CSRFToken	—	403 CSRF failure		
ST-SEC-02	System	SEC-005	Cross-site form post	Logged in in one tab	Submit a form from another origin to /api/stock/transactions/	—	Rejected; no stock change		
ST-SEC-03	System	SEC-001, SEC-003	Automated scans	Test build	Bandit, pip-audit, OWASP ZAP baseline scan	—	No high-severity findings left open		
ST-SEC-04	System	SEC-006, FR-019	Audit trail review	After running the suites	Admin opens Audit log, filters by action	—	Failed logins, 403s, deletions and stock moves all present with actor and time		

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
ST-SEC-05	System	SEC-004	Session cookie flags and timeout	Deployed over HTTPS	Inspect cookie; stay idle 31 min	—	Secure, HttpOnly, SameSite=Lax; redirected to login after timeout		

6. Test Environment

Details	
Hardware	Team laptops (Windows / Linux, 8 GB RAM or more) for development and system tests; a Linux VM / laptop running Docker for the shared test deployment and Jenkins
Software	ESIMS v1.0 build; Python 3.12, Django 5, DRF; PostgreSQL 14; nginx with a self-signed HTTPS certificate; latest Chrome, Edge and Firefox
Tools	pytest, pytest-django, coverage.py (unit / integration); Selenium (UI automation); Postman (API); Locust (performance); Bandit, pip-audit, SonarQube, OWASP ZAP (security); Jenkins (CI); GitHub Issues (defects)

Environment	Setup	Used for
Developer machine	Python 3.12, Django 5, SQLite	Unit tests while coding
CI (Jenkins)	Docker containers: web + PostgreSQL 14	Unit + integration on every push / PR, coverage, scans
Test deployment	Docker Compose: nginx (HTTPS) + web + PostgreSQL 14, seeded data	System, security, performance, cross-browser, UAT

Test data. A fixture script (`manage.py seed_test_data`) loads the same data every time, so results can be repeated. The performance run uses a larger generated set.

Set	Contents
Users	admin1 (Admin), staff1, staff2 (disabled), plus one locked-out account
Categories	Microcontrollers, Sensors, Passive Components, Power Supplies, Cables & Connectors
Suppliers	Robu.in, Element14 India, Local Wholesaler (no components linked)
Components	About 60 items covering all three statuses, e.g. Arduino Uno R3 (qty 25, reorder 5), HC-SR04 Ultrasonic Sensor (qty 3, reorder 10), 10kΩ resistor pack (qty 0, reorder 20)
Performance set	5,000 generated components across all categories

Test data matrix.

Scenario	Data	Expected
Valid	Stock-out 3 from qty 10	Accepted, qty 7
Boundary	Stock-out 10 from qty 10	Accepted, qty 0, OUT_OF_STOCK
Boundary	Stock-out 11 from qty 10	Rejected, qty unchanged
Boundary	qty = reorder_level	LOW_STOCK
Boundary	qty = reorder_level + 1	IN_STOCK
Invalid	Quantity 0 or negative	Rejected
Invalid	Duplicate SKU (any case / spacing)	Rejected
Boundary	Name of 100 chars / 101 chars	Accepted / rejected
Security	Staff session on an admin endpoint	403
Security	' OR 1=1 -- in search; <script> in name	No effect / shown as text
Concurrency	Two stock-outs of 4 on qty 5	One succeeds, final qty 1

7. Test Schedule

We work in two-week sprints. Unit and integration tests are written in the same sprint as the feature, so testing runs the whole way through rather than only at the end. The dates below are our plan and will be updated on the GitHub project board each sprint.

Milestone	Date	Testing work
Test plan baseline	03-Oct-2026	This document submitted with the SRS and SAD
Environment setup	05-Oct-2026 to 16-Oct-2026	Jenkins pipeline with pytest and coverage; Docker Compose test deployment; seed script (Sprint 1)
Test case design (detailed scripts)	by 16-Oct-2026	pytest skeletons and Selenium scripts for the cases in Section 13.3
Test execution start	19-Oct-2026	Sprint 2: auth, users, catalogue unit + integration tests
Stock, reports, dashboard	02-Nov-2026 to 13-Nov-2026	Sprint 3: stock tests incl. IT-STK-02 on PostgreSQL; report, export and dashboard tests
System, security and performance	16-Nov-2026 to 27-Nov-2026	Sprint 4: full system run, Section 5.1 security tests, ZAP scan, PERF-01, cross-browser
Test execution end	27-Nov-2026	All cases executed; Actual Result and Pass / Fail filled in
UAT	30-Nov-2026 to 02-Dec-2026	Usability walkthrough and main flows with a non-team user
Test summary report	04-Dec-2026	Coverage report, defect summary and RTM status for the final report

8. Test Deliverables

- Software Test Plan (this document).
- Test cases (Section 13.3), kept in sync with the pytest and Selenium code in the repository `tests/` folders.
- Test scripts: pytest modules, Selenium scripts, Postman collection, Locust file.
- Test data: `seed_test_data` command and the generated performance set.
- Test execution logs and the Jenkins console output for each run.
- pytest and coverage reports (HTML + summary).
- Security reports: Bandit, pip-audit, SonarQube and OWASP ZAP.
- Defect reports (GitHub Issues with the `bug` label).
- Test summary report, including the completed RTM.

9. Roles and Responsibilities

We are a team of two, so each of us takes more than one role. The rule we keep is that nobody is the only reviewer of their own module.

Role	Name	Responsibility
QA Lead	Ayaan Ahmed	Owens this plan, security testing (Section 5.1) and the RTM; coordinates test runs
Test Engineer	C. Harshal Rajesh	Writes and runs tests for stock, reports and dashboard; PERF-01; runs system tests for accounts and catalogue
Test Engineer	Ayaan Ahmed	Writes and runs tests for accounts, catalogue, categories and suppliers; runs system tests for stock, reports and dashboard
Developer	C. Harshal Rajesh, Ayaan Ahmed	Each fixes defects in the modules they own; triage together at the start of each sprint
CI / environment	C. Harshal Rajesh	Jenkins pipeline, Docker test deployment, PostgreSQL, scan stages

Role	Name	Responsibility
Product Owner	C. Harshal Rajesh	Accepts the UAT results and signs off release readiness
Course Faculty	Dr. Sapna V M	Reviews the plan and the final validation evidence

10. Risks and Mitigation

Risk	Mitigation
SQLite does not lock rows the way PostgreSQL does, so a concurrency bug could slip through	Run IT-STK-02 only against PostgreSQL in CI and in the test deployment
With two people, one person being away stalls a module's testing	Either of us can run the other's suite from the README; everything also runs automatically in Jenkins
Late changes to the API contract break integration tests	Contract changes go through a PR reviewed by both of us; the Postman collection is updated in the same PR
Test data drifts between runs, so results can't be repeated	The seed script recreates the database before every system test run
Not enough time at the end, so non-functional tests get skipped	Security and performance tests are in Sprint 4, not the last week
Test deployment down (laptop / VM unavailable)	Docker Compose lets us bring the same environment up on either laptop in a few minutes

11. Assumptions & Dependencies

- The SRS v1.0 requirements stay stable during the sprints. Any change goes through a PR that also updates this plan and the RTM.
- Each module's unit tests are written by its owner in the same sprint as the feature.
- A Docker-capable machine is available for Jenkins and the test deployment.
- PostgreSQL 14 is available in CI for the concurrency and transaction tests.
- SonarQube, Bandit, pip-audit, OWASP ZAP and Locust stay free to use for student projects.
- A person outside the team is available for the usability walkthrough and UAT.

12. Suspension & Resumption Criteria

Suspend testing if:

- The build does not start, login is broken, or database migrations fail. These block nearly every other test.
- More than 30% of the planned cases for the sprint are blocked by open defects.
- The test deployment is unavailable for more than 4 hours.

Resume testing when:

- The blocking defect is fixed and merged, and the smoke tests ST-AUTH-01 and ST-INV-02 pass again.
- The environment is back up and the seed data has been reloaded.

13. Test Case Management & Traceability

13.1 How test cases are managed

Each test case has a unique ID that shows its level and module: UT- unit, IT- integration, ST- system, PERF- performance, followed by the module (AUTH, USR, INV, CAT, SUP, STK, RPT, DSH, SEC, NFR). The same ID is used in the pytest function name or Selenium script, for example

`test_UT_STK_01_stockout_above_available_is_refused`, so a failing build points straight back to this plan. Defects are logged as GitHub Issues that quote the failing test ID, and the fix goes in through a pull request that references the issue.

13.2 Requirements Traceability Matrix

Every requirement in the SRS maps to at least one test case. Status: N = not run yet, P = passed, A = attention (failed or blocked). All are N until execution starts. The design side of the chain (requirement → component → design → code) is in SAD Section 3.8.

Req ID	Requirement short	Test cases	Levels	Status
FR-001	Admin creates user accounts	UT-USR-01, UT-USR-02, ST-USR-01	Unit, System	N
FR-002	Authenticate user	UT-AUTH-01, UT-AUTH-02, UT-AUTH-04, IT-AUTH-01, ST-AUTH-01, ST-AUTH-02	Unit, Integration, System	N
FR-003	Role-based access	ST-AUTH-01, UT-USR-05, IT-USR-01	Unit, Integration, System	N
FR-004	Add component	UT-INV-01, UT-INV-04, UT-INV-06, IT-INV-01, ST-INV-01	Unit, Integration, System	N
FR-005	Reject duplicate SKU	UT-INV-02, UT-INV-03, IT-INV-01, UT-INV-09	Unit, Integration	N
FR-006	Edit component	UT-INV-07, IT-INV-03, ST-INV-03, ST-INV-04	Unit, Integration, System	N
FR-007	Soft-delete component (Admin)	IT-INV-04, UT-INV-11, UT-INV-12, IT-INV-06, IT-INV-08, IT-INV-09, ST-INV-05	Unit, Integration, System	N
FR-008	Paginated, searchable list	UT-INV-14, UT-INV-15, IT-INV-11, ST-INV-02	Unit, Integration, System	N
FR-009	Filter by category / supplier / status	UT-INV-16, UT-INV-17, IT-INV-10, IT-INV-13, ST-INV-02	Unit, Integration, System	N
FR-010	Record stock-in	UT-STK-03, UT-STK-04, ST-STK-01	Unit, System	N
FR-011	Stock-out, never below zero	UT-STK-01, UT-STK-04, UT-STK-05, IT-STK-01, ST-STK-01	Unit, Integration, System	N
FR-012	Automatic Low / Out of Stock status	UT-INV-18, UT-INV-08, UT-STK-02, UT-STK-05, IT-STK-04, ST-STK-01	Unit, Integration, System	N
FR-013	Manage categories	UT-CAT-01, UT-CAT-02, IT-CAT-02, ST-CAT-01	Unit, Integration, System	N
FR-014	Manage suppliers	UT-SUP-01, ST-CAT-01	Unit, System	N

Req ID	Requirement short	Test cases	Levels	Status
FR-015	Availability report with filters	UT-RPT-01, UT-RPT-02, IT-RPT-01, ST-RPT-01	Unit, Integration, System	N
FR-016	Export report CSV / PDF	UT-RPT-03, IT-RPT-02, IT-RPT-03, ST-RPT-01	Unit, Integration, System	N
FR-017	Dashboard KPIs and chart	UT-DSH-01, UT-DSH-02, UT-DSH-03, UT-DSH-04, IT-DSH-01, IT-DSH-02, ST-DSH-01	Unit, Integration, System	N
FR-018	Record every stock transaction	UT-INV-10, UT-INV-13, IT-STK-01, IT-STK-03	Unit, Integration	N
FR-019	Audit log of security events	IT-AUTH-03, IT-INV-03, IT-INV-06, ST-SEC-04	Integration, System	N
NFR-001	95% of list / search ≤ 2 s	PERF-01, ST-DSH-02	System	N
NFR-002	$\geq 99\%$ availability in store hours	ST-NFR-03	System	N
NFR-003	Add a component in ≤ 3 screens	ST-INV-01, ST-NFR-02	System	N
NFR-004	Concurrent stock-outs never go negative	IT-STK-02	Integration	N
NFR-005	Business logic in identifiable modules	ST-NFR-04	System	N
NFR-006	5,000 components, 10 concurrent users	PERF-01	System	N
NFR-007	Latest Chrome, Edge, Firefox	ST-NFR-01	System	N
NFR-008	Testable core services	UT-STK-02	Unit	N
SEC-001	Salted password hashes	UT-AUTH-03, UT-USR-03, ST-SEC-03	Unit, System	N
SEC-002	RBAC blocks Staff from Admin functions	IT-INV-07, ST-INV-05, IT-CAT-01, IT-DSH-01, IT-USR-01, ST-SEC-01	Integration, System	N
SEC-003	Input validated against SQLi / XSS	IT-INV-02, IT-INV-05, IT-INV-12, UT-RPT-04, ST-SEC-03	Unit, Integration, System	N
SEC-004	Sessions expire; bad credentials rejected	IT-AUTH-02, IT-AUTH-04, UT-USR-04, ST-SEC-05	Unit, Integration, System	N
SEC-005	CSRF protection	IT-SEC-01, ST-SEC-02	Integration, System	N
SEC-006	Security events logged	IT-AUTH-03, IT-USR-01, ST-SEC-04	Integration, System	N

Business rules: BR-1 (no negative quantity) $\rightarrow$ UT-STK-01, IT-STK-02 · BR-2 (unique SKU) $\rightarrow$ UT-INV-02, UT-INV-03 · BR-3 (reorder ≥ 0) $\rightarrow$ UT-INV-05 · BR-4 (stock-out only if enough) $\rightarrow$ UT-STK-01 · BR-5 (Admin-only deletes) $\rightarrow$ IT-INV-07, IT-CAT-01 · BR-6 (no hard delete when in use) $\rightarrow$ UT-CAT-03, UT-SUP-02, IT-CAT-03 · BR-7 (derived status) $\rightarrow$ UT-STK-02, UT-INV-08.

13.3 Test Cases

99 test cases in total: 8 to 10 for each use case across unit, integration and system level, plus the cross-cutting security cases in Section 5.1 and the non-functional cases at the end of this section. **Actual result** and **Pass / Fail** are left blank on purpose. They are filled in when each case is run against the build, and the completed sheet goes into the final validation report.

13.3.1 UC-01 Login / Authenticate

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
UT-AUTH-01	Unit	FR-002	Valid credentials authenticate the user	User staff1 exists, active	Call AuthService.authenticate()	staff1 / Store@2026	Returns user with role STAFF		
UT-AUTH-02	Unit	FR-002	Wrong password is rejected	staff1 exists	Call authenticate() with wrong password	staff1 / wrong123	Returns None; no session created		
UT-AUTH-03	Unit	SEC-001	Password is stored as a salted hash	—	Create two users with the same password; read DB rows	Both use Store@2026	Hashes start with pbkdf2_sha256\$, differ from each other and from the plain text		
UT-AUTH-04	Unit	FR-002	Disabled account cannot log in	staff2 has is_active = false	Call authenticate()	staff2 / valid password	Login refused, message 'Account disabled'		
IT-AUTH-01	Integration	FR-002	Login API returns session and role	staff1 exists	POST /api/auth/login/	Valid JSON body	200, session cookie set, body has role STAFF		
IT-AUTH-02	Integration	SEC-004	Expired session is rejected	Logged in; session idle > 30 min (clock moved forward)	GET /api/components /	Expired session cookie	401; user sent back to login page		
IT-AUTH-03	Integration	SEC-006, FR-019	Failed logins are written to the audit log	—	POST login 3 times with a wrong password	staff1 / bad	3 AuditLog rows, action LOGIN_FAILED, with username, IP and time		
IT-AUTH-04	Integration	SEC-004	Account locks after 5 failed attempts	—	6 wrong logins, then 1 correct login within 15 min	staff1	6th attempt and the correct one both refused (423) until 15 min pass		
ST-AUTH-01	System	FR-002, FR-003	Role-based landing page and logout through the UI	Admin and staff accounts exist	Log in as admin, log out, log in as staff, log out, press Back	admin1, staff1	Admin lands on Dashboard, staff on Inventory list; after logout, Back shows no protected data		
ST-AUTH-02	System	FR-002	Blank fields are caught on the form	On login page	Click Login with empty fields	"	Inline 'required' errors; no request sent		

13.3.2 UC-02 Add Component

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
UT-INV-01	Unit	FR-004	Valid component is created	Category Microcontrollers and supplier Robu.in exist	CatalogueService.create()	Arduino Uno R3, MCU-ARD-UNO, ₹650.00, qty 25, reorder 5	Saved; status IN_STOCK		
UT-INV-02	Unit	FR-005	Duplicate SKU is rejected	MCU-ARD-UNO already exists	create() with same SKU	SKU MCU-ARD-UNO	DuplicateSKU raised; component count unchanged		
UT-INV-03	Unit	FR-005	SKU is normalised before the uniqueness check	MCU-ARD-UNO exists	create() with lower case + spaces	'mcu-ard-uno '	Treated as duplicate and rejected		
UT-INV-04	Unit	FR-004	Negative price is rejected	—	Validate serializer	unit_price = -10	Field error on unit_price		

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
UT-INV-05	Unit	BR-3	Negative reorder level is rejected	—	Validate serializer	reorder_level = -1	Field error on reorder_level		
UT-INV-06	Unit	FR-004	Mandatory fields are enforced	—	Validate serializer with name and category missing	{sku only}	Errors for name and category		
UT-INV-18	Unit	FR-012	Initial status is derived from the opening quantity	—	create() with qty 0 and with qty 3 (reorder 5)	qty 0 / qty 3	OUT_OF_STOCK / LOW_STOCK		
IT-INV-01	Integration	FR-004, FR-005	Create via API, then a duplicate via API	Logged in as staff	POST /api/components / twice with same SKU	Valid body	201 first time and AuditLog COMPONENT_CREATED; 400 sku 'already exists' second time		
IT-INV-02	Integration	SEC-003	Script in the name is stored and shown as text	Logged in	POST component, then open the list page	name = <script>alert(1)</script>	Name appears escaped; no script runs		
ST-INV-01	System	FR-004, NFR-003	Add a component through the UI in at most 3 screens	Staff account	Login → Inventory list → Add → Save	Arduino Uno R3 data	Item appears in the list with green In Stock badge; 3 screens used		

13.3.3 UC-03 Edit Component

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
UT-INV-07	Unit	FR-006	Price and reorder level update	Component exists	CatalogueService.update()	price 650 → 699	New values saved		
UT-INV-08	Unit	FR-012	Raising reorder level re-derives status	qty 8, reorder 5 (IN_STOCK)	update() reorder level	reorder 5 → 10	Status becomes LOW_STOCK		
UT-INV-09	Unit	FR-005	Editing SKU to an existing one is rejected	SKUs A and B exist	Change B's SKU to A	sku = A	DuplicateSKU raised; B unchanged		
UT-INV-10	Unit	FR-018	Quantity cannot be changed through edit	qty 25	update() with quantity in payload	quantity = 999	Quantity stays 25		
IT-INV-03	Integration	FR-006, FR-019	Edit via API is audited	Logged in as staff	PATCH /api/components /{id}/	{unit_price: 699}	200; AuditLog COMPONENT_UPDATED with old and new values		
IT-INV-04	Integration	FR-007	Soft-deleted component cannot be edited	Component soft-deleted	PATCH it	{name: 'x'}	404		
IT-INV-05	Integration	SEC-003	Wrong data type is rejected	Logged in	PATCH with a non-numeric price	unit_price = 'abc'	400 with field error		
ST-INV-03	System	FR-006	Edit through the UI	Component exists	Open Edit, change name and price, Save	New name, ₹699	Form was pre-filled; list shows new values and a success message		
ST-INV-04	System	FR-006	Cancel leaves data unchanged	Component exists	Open Edit, change fields, click Cancel	—	No change saved		

13.3.4 UC-04 Delete Component

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
UT-INV-11	Unit	FR-007	Soft delete marks the row instead of removing it	Component exists	component.soft_delete(admin)	—	is_deleted = true, deleted_at set; row still in DB		
UT-INV-12	Unit	FR-007	Deleted items are hidden from default queries	1 deleted, 2 active	Component.active.all()	—	Returns only the 2 active ones		
UT-INV-13	Unit	FR-018	Transaction history survives the delete	Component with 3 transactions	Soft delete, then count transactions	—	Still 3 transactions		

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
IT-INV-06	Integration	FR-007, FR-019	Admin deletes via API	Logged in as admin	DELETE /api/components/{id}/	—	204; AuditLog COMPONENT_DELETED		
IT-INV-07	Integration	SEC-002	Staff cannot delete	Logged in as staff	DELETE /api/components/{id}/	—	403; component unchanged; AuditLog UNAUTHORIZED_ACCESS		
IT-INV-08	Integration	FR-007	Deleting an unknown ID	Logged in as admin	DELETE random UUID	—	404		
IT-INV-09	Integration	FR-007	No stock movement on a deleted component	Component soft-deleted	POST stock IN for it	qty 5	404; quantity unchanged		
ST-INV-05	System	FR-007, SEC-002	Delete from the UI with confirmation	Admin and staff accounts	Admin: Delete → Confirm. Staff: open the same list	—	Item disappears for admin; staff never sees a Delete button		

13.3.5 UC-05 Search / View Inventory

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
UT-INV-14	Unit	FR-008	Name search is case-insensitive substring	Arduino Uno R3, Arduino Nano exist	search('uno')	'uno'	Only Arduino Uno R3		
UT-INV-15	Unit	FR-008	SKU search matches exactly	MCU-ARD-UNO exists	search('MCU-ARD-UNO')	—	1 result		
UT-INV-16	Unit	FR-009	Filter by category	Mixed categories	filter(category=Sensors)	—	Only sensors returned		
UT-INV-17	Unit	FR-009	Filter by status	Some low-stock items	filter(status=LOW_STOCK)	—	Only LOW_STOCK items		
IT-INV-10	Integration	FR-009	Combined filters via API	Seed data	GET /api/components/?category=&supplier=&status=	Sensors + Robu.in + LOW	Only rows matching all three		
IT-INV-11	Integration	FR-008	Pagination	60 components	GET page=3	page size 25	count = 60; page 3 has 10 results		
IT-INV-12	Integration	SEC-003	SQL injection string in search	Seed data	GET ?search=	' OR 1=1 --	200 with 0 results; no error; table intact		
IT-INV-13	Integration	FR-009	Invalid filter value	—	GET ? status=MAYBE	—	400 with a clear message		
ST-INV-02	System	FR-008, FR-009	List screen shows quantities and coloured badges	Seed data with all 3 statuses	Open list, search, filter, clear filters	—	Badges green / amber / red; 'No components match' when empty		
PERF-01	System	NFR-001, NFR-006	List / search response time under load	5,000 components	Locust: 10 users, 5 min, list + search	—	95th percentile ≤ 2 s; no errors		

13.3.6 UC-06 Record Stock-in / Stock-out

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
UT-STK-01	Unit	FR-011	Stock-out above available quantity is refused	qty 10	StockService.record(OUT, 11)	11	InsufficientStock; qty stays 10; no transaction row		
UT-STK-02	Unit	FR-012, NFR-008	Availability boundaries	—	classify(qty, reorder)	(0,5) (3,5) (5,5) (6,5)	OUT, LOW, LOW, IN		
UT-STK-03	Unit	FR-010	Stock-in increases quantity	qty 10	record(IN, 15)	15	qty 25; transaction saved		
UT-STK-04	Unit	FR-010, FR-011	Zero or negative quantity is refused	qty 10	record(IN, 0); record(OUT, -3)	0, -3	Validation error both times		
UT-STK-05	Unit	FR-011, FR-012	Stock-out of exactly all units	qty 4	record(OUT, 4)	4	qty 0, status OUT_OF_STOCK		

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
IT-STK-01	Integration	FR-011, FR-018	Stock-out via API updates quantity and logs the movement	qty 10, staff logged in	POST /api/stock/transactions/	{OUT, 3}	201; qty 7; StockTransaction has component, type, qty, user, timestamp		
IT-STK-02	Integration	NFR-004	Two simultaneous stock-outs cannot oversell	qty 5 on PostgreSQL	Two threads each POST OUT 4 at the same time	4 + 4	Exactly one 201 and one 409; final qty 1		
IT-STK-03	Integration	FR-018	Failure part-way rolls everything back	qty 10	Force an error after the transaction insert	OUT 2	Neither quantity nor transaction is saved		
IT-STK-04	Integration	FR-012	Low-stock alert only when the status changes	qty 7, reorder 5	OUT 3, then OUT 1	—	Notifier called once (7→4), not on 4→3		
ST-STK-01	System	FR-010 – FR-012	Stock-out through the UI	Component qty 7, reorder 5	Open Stock form, pick SKU, OUT, qty 3, Submit	—	Success message; badge turns amber; history shows the entry with user name		

13.3.7 UC-07 Manage Categories / Suppliers

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
UT-CAT-01	Unit	FR-013	Category names are unique, case-insensitive	Sensors exists	create('sensors')	—	Duplicate error		
UT-CAT-02	Unit	FR-013	Unused category can be deleted	Category with 0 components	delete()	—	Deleted		
UT-CAT-03	Unit	BR-6	In-use category cannot be deleted	Category with 12 components	delete()	—	InUseError 'used by 12 components'		
UT-SUP-01	Unit	FR-014	Supplier email is validated	—	create supplier	contact_email = 'robu@'	Field error on email		
UT-SUP-02	Unit	BR-6	In-use supplier cannot be deleted	Supplier linked to components	delete()	—	InUseError		
IT-CAT-01	Integration	SEC-002	Staff cannot create a category	Staff logged in	POST /api/categories/	{name: 'Cables'}	403		
IT-CAT-02	Integration	FR-013	Renaming a category shows everywhere	Admin logged in	PATCH category, then GET components	'Sensors' → 'Sensors & Modules'	Components list shows the new name		
IT-CAT-03	Integration	BR-6	Delete in-use category via API	Admin logged in	DELETE /api/categories/{id}/	—	409 with the in-use message		
ST-CAT-01	System	FR-013, FR-014	Manage categories and suppliers through the UI	Admin logged in	Add, edit, try to delete an in-use one, delete an unused one	—	All work; in-use delete button is disabled with a tooltip		

13.3.8 UC-08 Generate Availability Report

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
UT-RPT-01	Unit	FR-015	Status filter	Mixed stock	ReportService.build(status=OUT)	—	Only qty 0 rows		
UT-RPT-02	Unit	FR-015	Deleted items are excluded	1 deleted component	build()	—	Deleted item not in rows or totals		
UT-RPT-03	Unit	FR-016	CSV has the right header and row count	20 components	export_csv()	—	Header: SKU, Name, Category, Supplier, Qty, Reorder, Status, Value; 20 data rows		
UT-RPT-04	Unit	SEC-003	CSV formula injection is neutralised	Name starts with '='	export_csv()	=HYPERLINK("x")	Cell written as '=HYPERLINK("x")		

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
IT-RPT-01	Integration	FR-015	Filtered report via API	Seed data	GET /api/reports/availability/ with category=Sensors, status=LOW_STOCK	—	200; rows match filters; totals correct; generated_at present		
IT-RPT-02	Integration	FR-016	PDF export	Seed data	GET .../export/?format=pdf	—	200 application/pdf; file opens; shows filters and timestamp		
IT-RPT-03	Integration	FR-016	Unsupported export format	—	GET .../export/?format=xls	—	400		
ST-RPT-01	System	FR-015, FR-016	Generate and export a report through the UI	Staff logged in	Set filters, Generate, Export CSV, open in a spreadsheet	—	Same rows and totals as on screen		

13.3.9 UC-09 View Dashboard

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
UT-DSH-01	Unit	FR-017	Component count ignores deleted items	5 active, 1 deleted	DashboardService.summary()	—	total_components = 5		
UT-DSH-02	Unit	FR-017	Stock value calculation	10 × ₹650.00 and 4 × ₹120.50	summary()	—	stock_value = ₹6,982.00		
UT-DSH-03	Unit	FR-017	Low and out-of-stock counts	2 LOW, 1 OUT, 3 IN	summary()	—	low_count = 2, out_count = 1		
UT-DSH-04	Unit	FR-017	Empty inventory	No components	summary()	—	All zeros; no error		
IT-DSH-01	Integration	FR-017, SEC-002	Summary API and role check	Admin and staff	GET /api/dashboard/summary/ as each	—	Admin 200 with all fields; staff 403		
IT-DSH-02	Integration	FR-017	KPIs follow stock changes	Item at qty 6, reorder 5	Stock-out 2, then GET summary	—	low_count goes up by 1; value goes down by 2 × price		
ST-DSH-01	System	FR-017	Dashboard through the UI	Admin logged in, seed data	Open Dashboard	—	4 KPI cards + bar chart by category; numbers match the report		
ST-DSH-02	System	NFR-001	Dashboard load time	5,000 components	Open Dashboard 10 times	—	Each load ≤ 2 s		

13.3.10 UC-10 Manage Users

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
UT-USR-01	Unit	FR-001	Create a staff user	—	create_user()	staff3, staff3@store.in	User saved with role STAFF, active		
UT-USR-02	Unit	FR-001	Duplicate username or email	staff1 exists	create_user() twice	same username; then same email	Error each time		
UT-USR-03	Unit	SEC-001	Weak password refused	—	create_user()	'12345'	Validation error (too short / too common)		
UT-USR-04	Unit	SEC-004	Disabling ends the user's sessions	staff1 logged in	disable(staff1)	—	is_active false; next request from staff1 gets 401		
UT-USR-05	Unit	FR-003	Admin cannot disable their own account	admin1 logged in	disable(admin1) by admin1	—	Error 'cannot disable your own account'		
IT-USR-01	Integration	FR-003, SEC-002, SEC-006	Staff cannot create users	Staff logged in	POST /api/users/	Valid body	403; AuditLog UNAUTHORIZED_ACCESS		
ST-SEC-01	System	SEC-002	Staff opens an admin URL directly	Staff logged in	Type /users/ and call /api/users/ by hand	—	403 page and 403 JSON; nothing shown		

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
ST-USR-01	System	FR-001	Admin creates a staff account who then logs in	Admin logged in	Users → New → Save; log out; log in as new user	staff3	New user lands on Inventory list		

13.3.11 Non-functional test cases

Test ID	Level	Req	Description	Precondition	Steps	Test data	Expected result	Actual result	Pass / Fail
ST-NFR-01	System	NFR-007	Cross-browser check	Test build	Run ST-INV-01, ST-STK-01, ST-RPT-01 on Chrome, Edge, Firefox	—	Same behaviour and layout on all three		
ST-NFR-02	System	NFR-003	Usability walkthrough	New user given a 2-minute intro	Ask them to add a component and record a stock-out	—	Done without help; ≤ 3 screens to add a component		
ST-NFR-03	System	NFR-002	Availability monitoring	Test deployment	Uptime check every 5 min over one week	—	≥ 99% during store hours		
ST-NFR-04	System	NFR-005	Coverage and structure check	CI run	pytest --cov on services; code review	—	≥ 80% line coverage on service modules; no business rules in views		

14. Test Metrics & Reporting

Metric	How we measure it	Target
Test cases executed	Executed / planned	100% by 27-Nov-2026
Pass rate	Passed / executed	100% High priority; ≥ 95% overall
Requirement coverage	Requirements with ≥ 1 passed case / total requirements (from the RTM)	100%
Code coverage	Line and branch coverage of service modules (coverage.py)	≥ 80% line
Defect density	Defects found per module per sprint	Tracked, no fixed target
Defect ageing	Days a defect stays open, by severity	Critical / High fixed within the sprint
Open security findings	High-severity findings from Bandit, pip-audit, SonarQube, ZAP	0 at release

Defect severity.

Severity	Meaning	Example
Critical	Data loss or security breach; blocks release	Stock goes negative; staff can delete users
High	Main feature broken, no workaround	Stock-out returns 500
Medium	Feature works but wrongly in some cases	CSV totals off by rounding
Low	Cosmetic	Long names overflow the badge column

Reports.

- Every push: Jenkins build result with test, coverage and scan summary.
- End of each sprint: short test status note in the sprint review (cases run, pass rate, open defects by severity).
- End of testing: Test Summary Report with the completed RTM, coverage report, security results and defect summary, used in the final validation report.

15. Approvals

Role	Name	Signature / Date
QA Lead	Ayaan Ahmed	
Dev Lead	C. Harshal Rajesh	
Product Owner	C. Harshal Rajesh	
Course Faculty	Dr. Sapna V M	